DISTRIBUTED SYSTEM · 4 เครื่อง

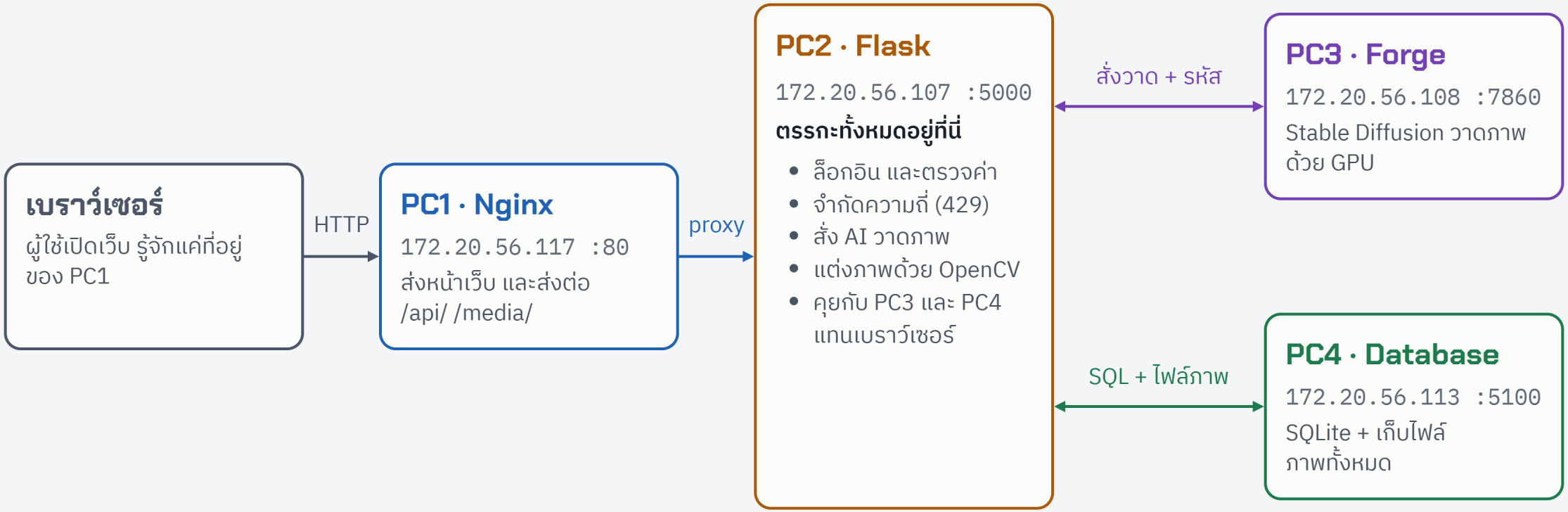
ImageProject

ระบบสร้างภาพด้วย AI และแต่งภาพ ที่แบ่งงานไว้บน 4 เครื่อง แล้วเชื่อมกันผ่าน HTTP

เบราว์เซอร์ → PC1 · Nginx → PC2 · Flask → PC3 · Forge AI + PC4 · Database

ภาพรวมระบบ

4 เครื่อง แต่ละเครื่องทำหน้าที่เดียว



ภาพรวมระบบ

หลักการ 4 ข้อที่ทุกส่วนใช้ร่วมกัน

01

เบราว์เซอร์คุยกับ PC1 ที่เดียว

หน้าเว็บ, /api/ และ /media/ ใช้ที่อยู่เดียวกัน Nginx ส่งต่อให้ จึงไม่ต้องตั้ง CORS

02

PC2 เป็นตัวกลาง

เบราว์เซอร์ไม่เคยคุยกับ PC3 หรือ PC4 ตรง ๆ ทุกคำขอผ่าน Flask ก่อนเสมอ

03

ข้อมูลและไฟล์ภาพอยู่ PC4

ฐานข้อมูลเก็บแค่ path เช่น outputs/1_1791.png ส่วนไฟล์จริงอยู่ในโฟลเดอร์ storage/

04

ทุก API ตอบเป็น JSON

ผิดพลาดเมื่อไหร่ ส่ง {"error": "ข้อความ"} พร้อม status 4xx/5xx แล้วหน้าเว็บเอาข้อความไปแสดง

ภาพรวมระบบ

เปิดระบบตามลำดับ PC4 → PC3 → PC2 → PC1

1 · PC4 Database

ดับเบิลคลิก start.bat

อ่าน .env ได้ DB_TOKEN สร้างตาราง และโฟลเดอร์ storage/

เปิดพอร์ต 5100

2 · PC3 Forge

Launch ใน Stability Matrix

ตั้ง --api --api-auth --listen ให้เครื่องอื่นสั่งวาดได้ แต่ต้องใส่รหัส

เปิดพอร์ต 7860

3 · PC2 Flask

ดับเบิลคลิก start.bat

อ่าน .env ได้ SECRET_KEY, DB_TOKEN และรหัส Forge

เปิดพอร์ต 5000

4 · PC1 Nginx

start-nginx.bat

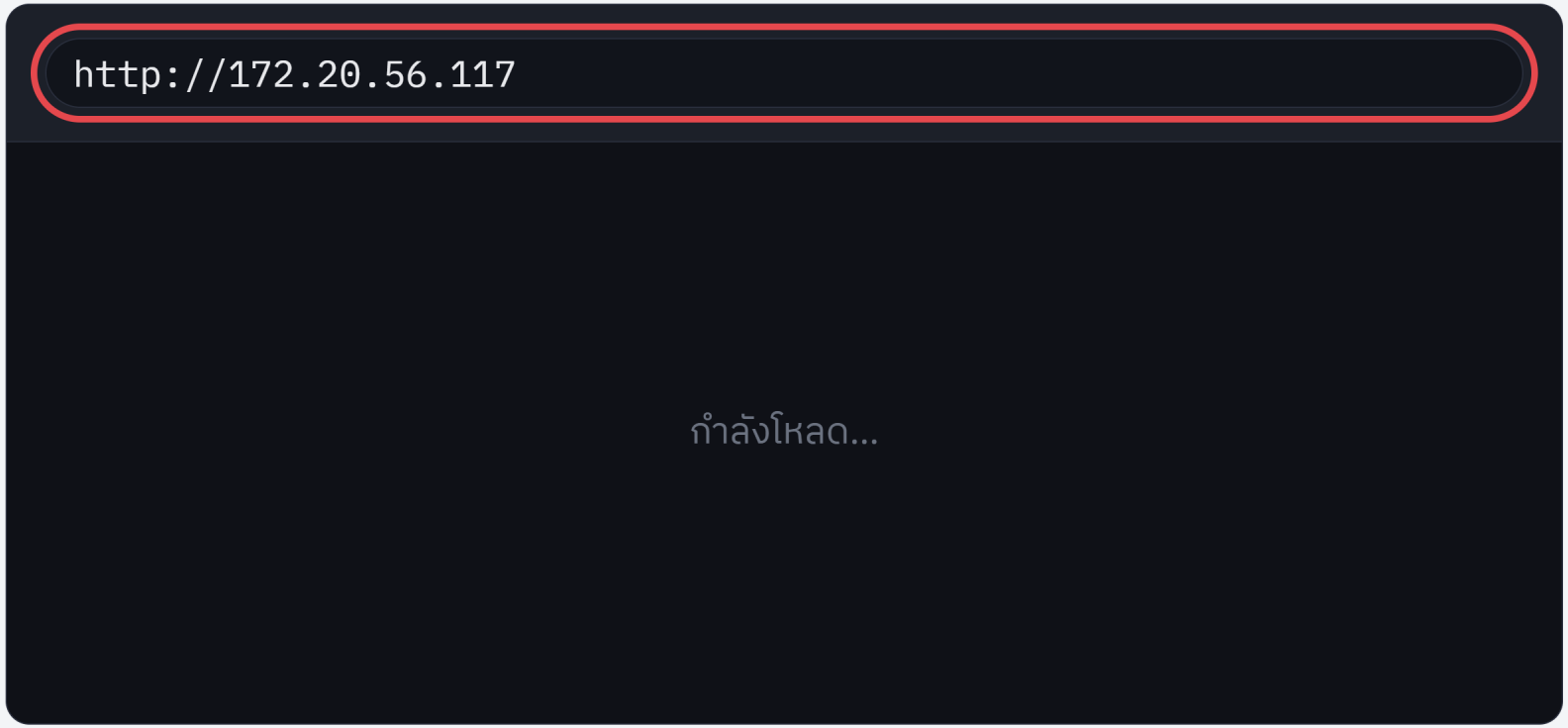
ตรวจ config ด้วย nginx -t ผ่านแล้วค่อยเปิด

เปิดพอร์ต 80 · ผู้ใช้เข้าเว็บได้

เปิดจากเครื่องที่อยู่หลังสุดก่อน เพราะเครื่องที่ถูกเรียก ต้องพร้อมก่อนเครื่องที่เป็นคนเรียก

เข้าเว็บ · สิ่งที่ผู้ใช้ทำ

ผู้ใช้พิมพ์ที่อยู่ของเว็บ



← พิมพ์ตรงนี้ แล้วกด Enter

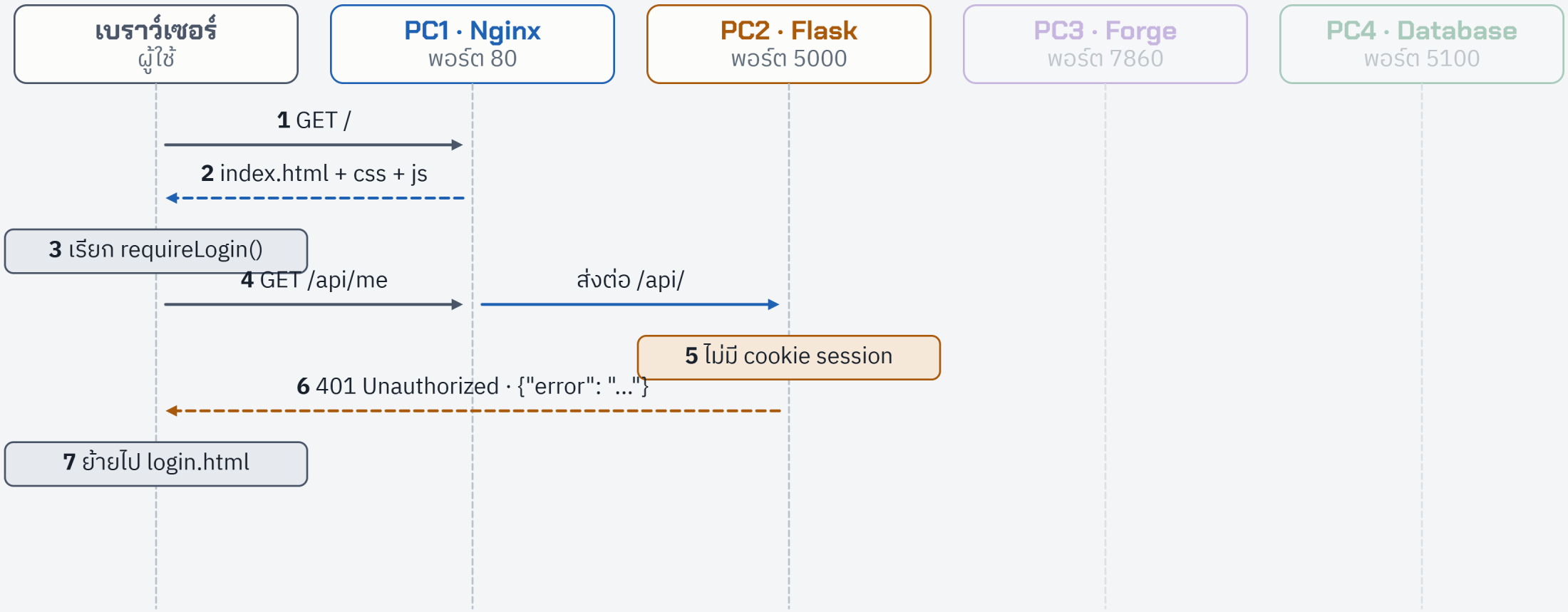
เปิดเว็บ

ผู้ใช้รู้จักแค่ที่อยู่ของ PC1 ไม่ต้องพิมพ์พอร์ต เพราะ 80 เป็นค่าเริ่มต้นของ http

ต่อไป: เบราว์เซอร์ส่งคำขอไปเครื่องไหนบ้าง →

เข้าเว็บ · ขั้นตอนการทำงาน

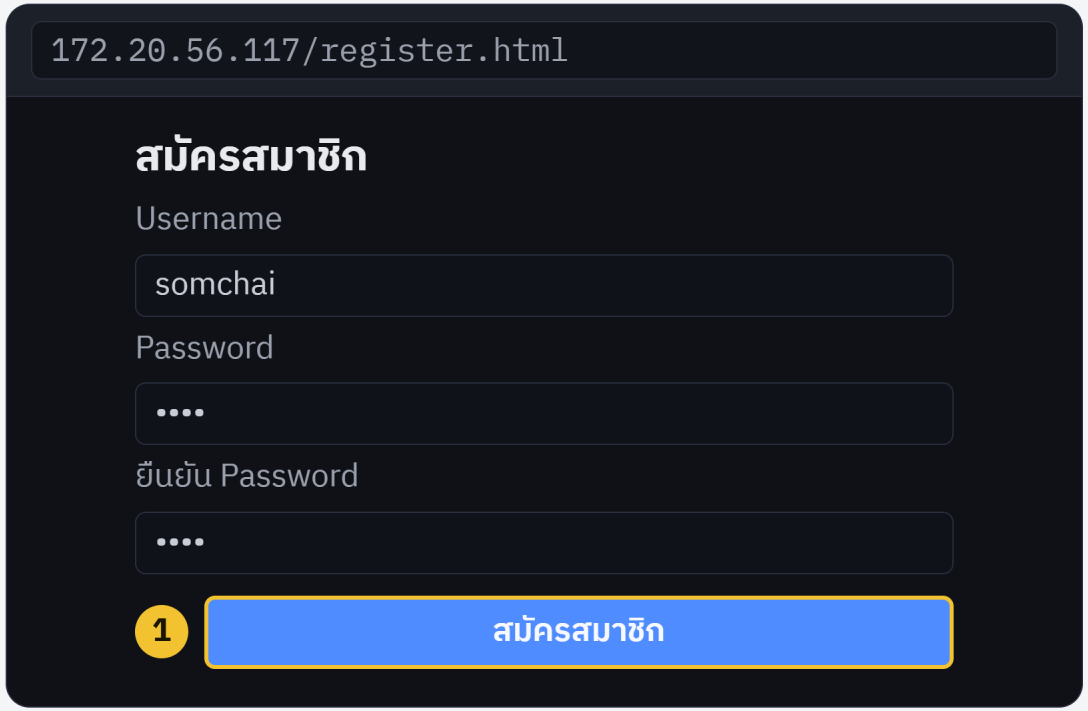
เปิดเว็บครั้งแรก ตอนยังไม่ได้ล็อกอิน



โค้ดของขั้นนี้ → ภาคผนวก C1 (Nginx) · C2 (requireLogin) · C5 (ตอบ 401)

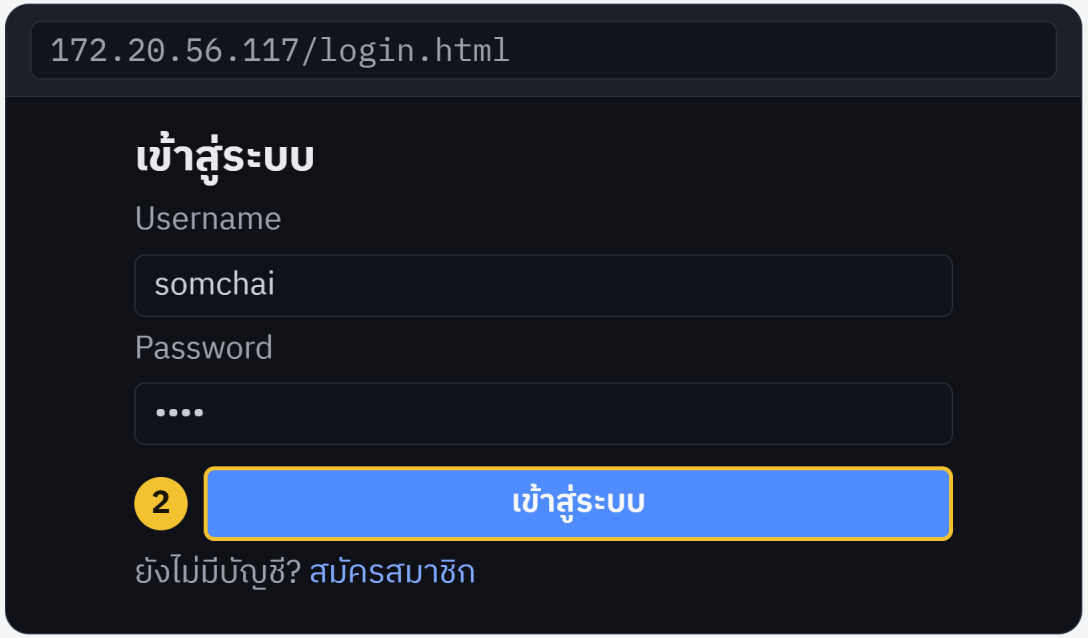
เข้าเว็บ · หน้าเว็บ

หน้าสมัครสมาชิก และหน้าเข้าสู่ระบบ



1 กด “สมัครสมาชิก”

auth.js ดัก submit ด้วย preventDefault() แล้วส่ง JSON ไปที่ POST /api/register

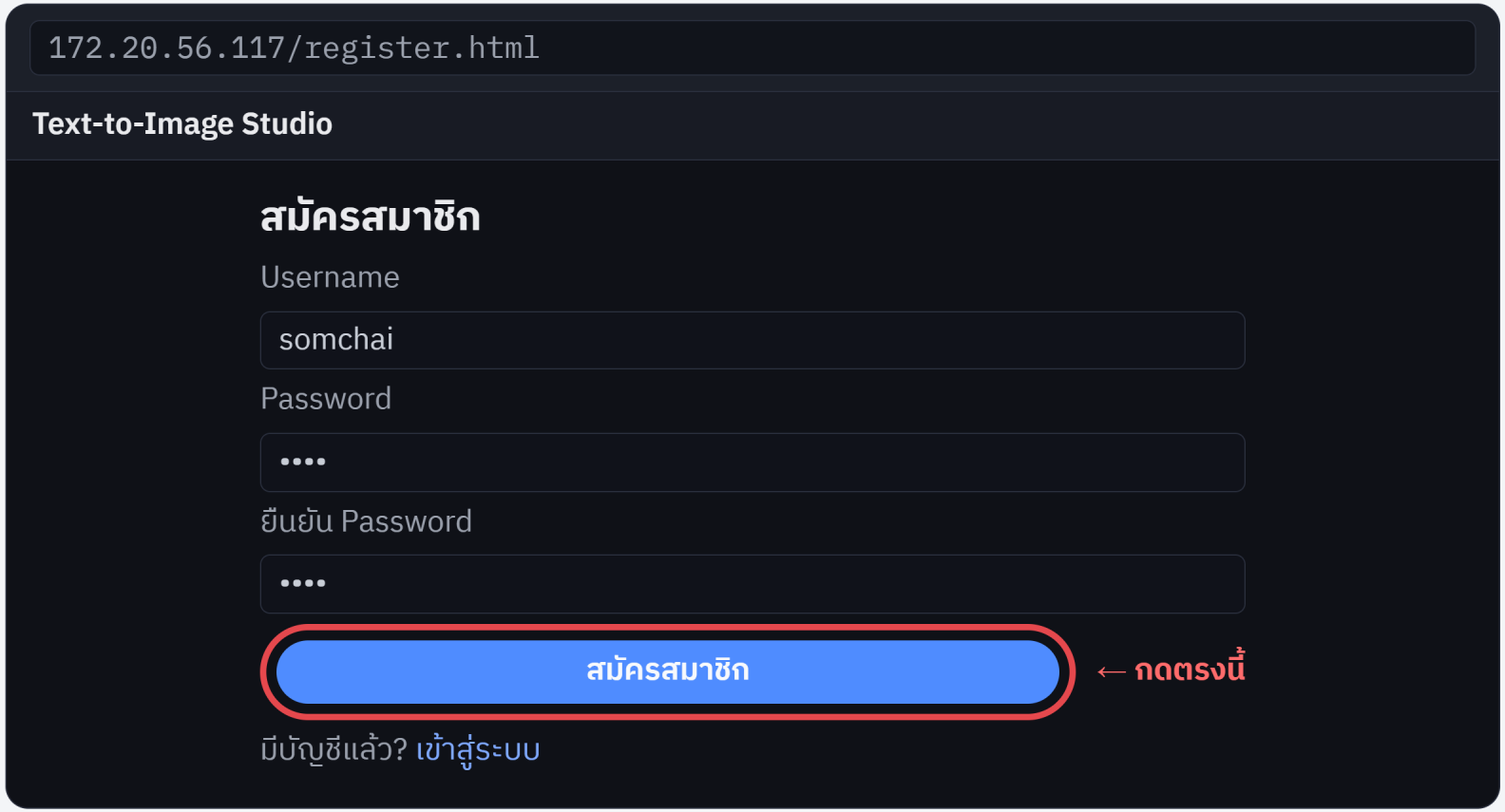


2 กด “เข้าสู่ระบบ”

ส่ง username / password ไปที่ POST /api/login ถูกต้องจะได้ cookie session กลับมา

เข้าเว็บ · สิ่งที่ผู้ใช้ทำ

ผู้ใช้กด “สมัครสมาชิก”



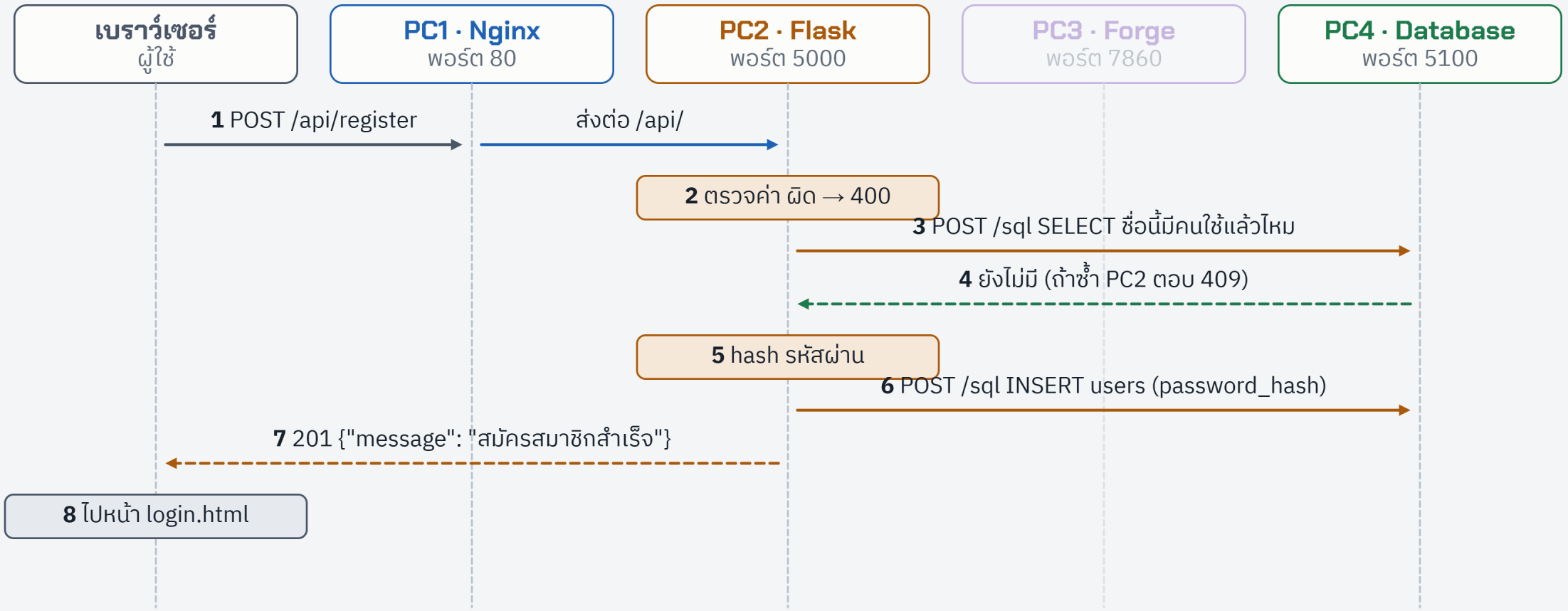
สมัครสมาชิก

auth.js ดัก submit แล้วส่ง JSON ไปที่ POST /api/register

ต่อไป: คำขอนี้วิ่งผ่านเครื่องไหนบ้าง →

เข้าเว็บ · ขั้นตอนการทำงาน

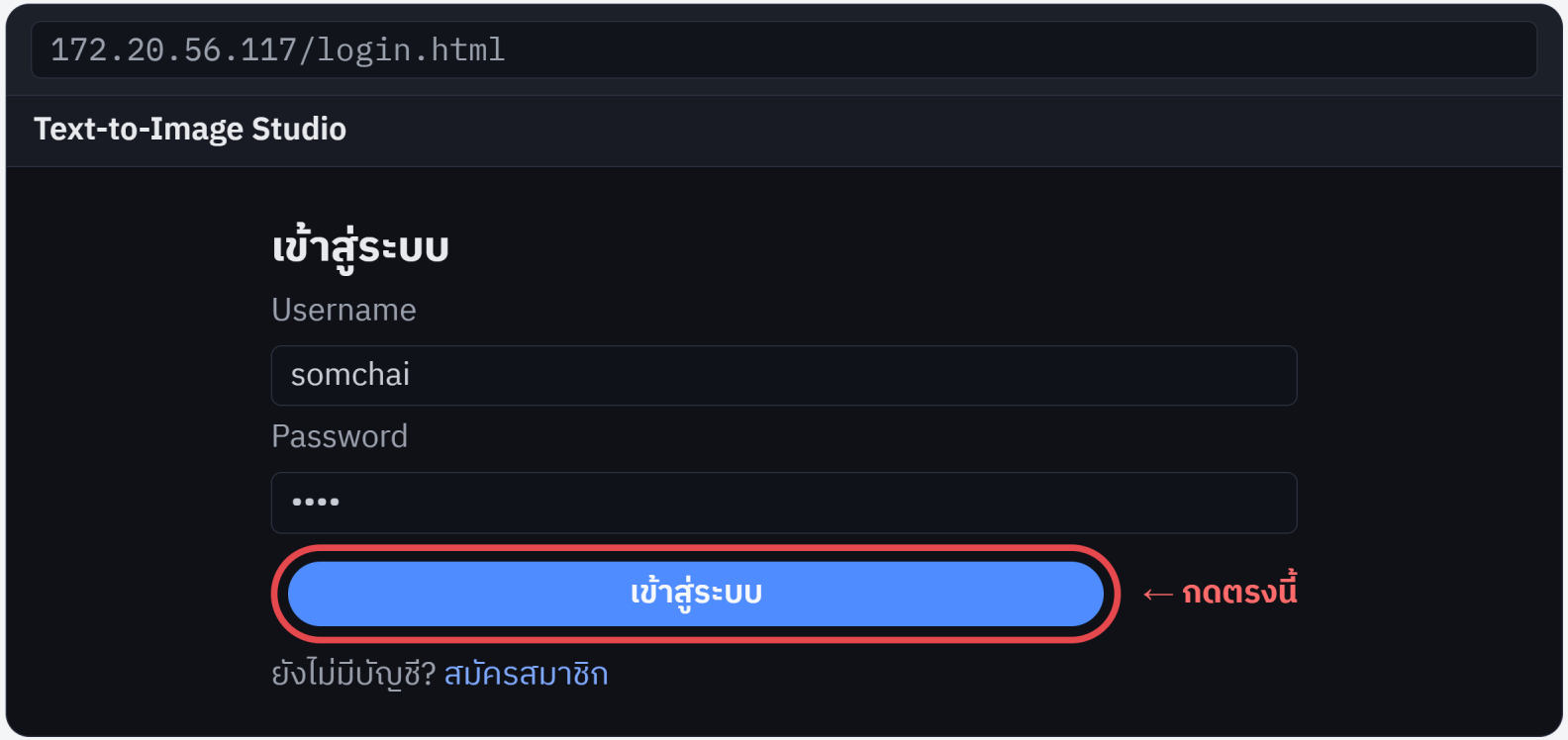
กด “สมัครสมาชิก”



โค้ดของขั้นนี้ → ภาคผนวก C3 (auth.js) · C4 (register / login) · C8 (ส่ง SQL ไป PC4)

เข้าเว็บ · สิ่งที่ผู้ใช้ทำ

ผู้ใช้กด “เข้าสู่ระบบ”



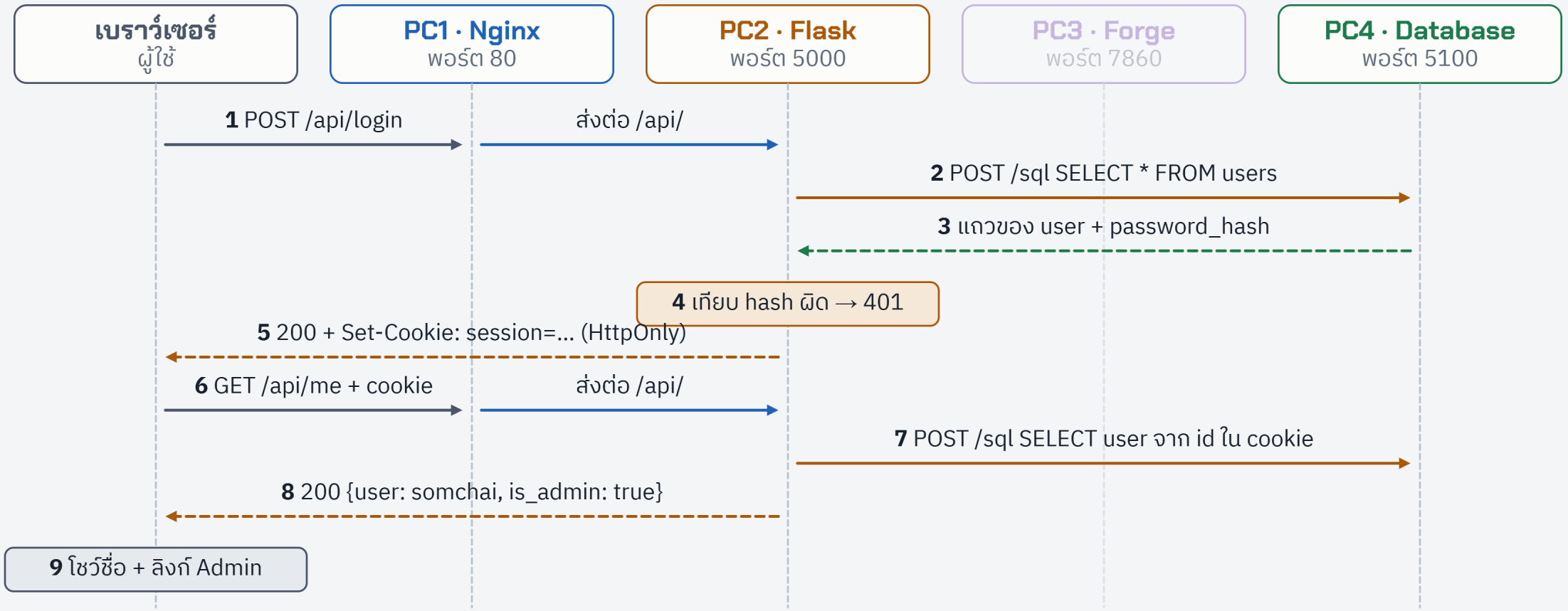
เข้าสู่ระบบ

ส่ง username และ password ไปที่ POST /api/login

ต่อไป: คำขอนี้วิ่งผ่านเครื่องไหนบ้าง →

เข้าเว็บ · ขั้นตอนการทำงาน

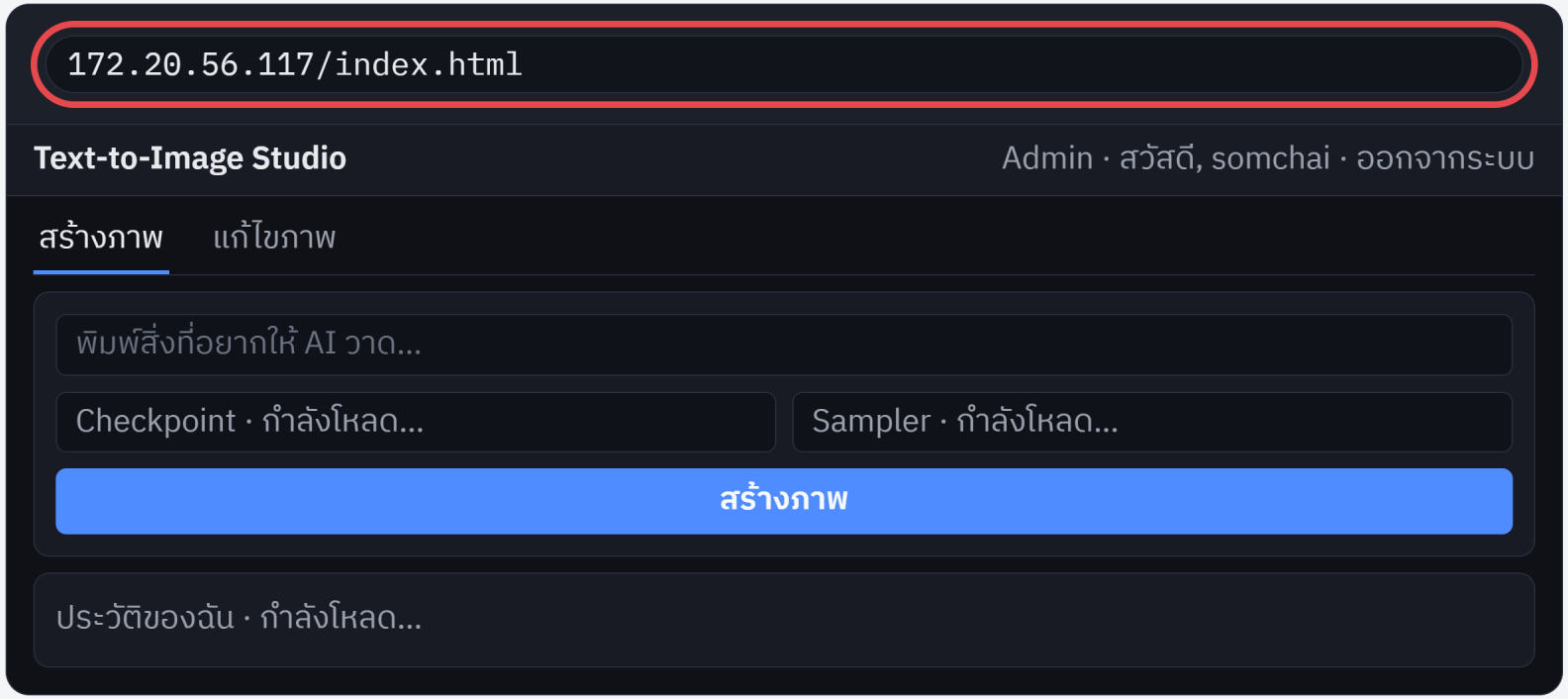
กด “เข้าสู่ระบบ”



โค้ดของขั้นนี้ → ภาคผนวก C4 (login) · C5 (cookie และ /api/me)

เข้าเว็บ · สิ่งที่ผู้ใช้เห็น

ล็อกอินสำเร็จ เปิดหน้าหลัก



← เบราว์เซอร์เปิดหน้านี้เอง หลังล็อกอิน

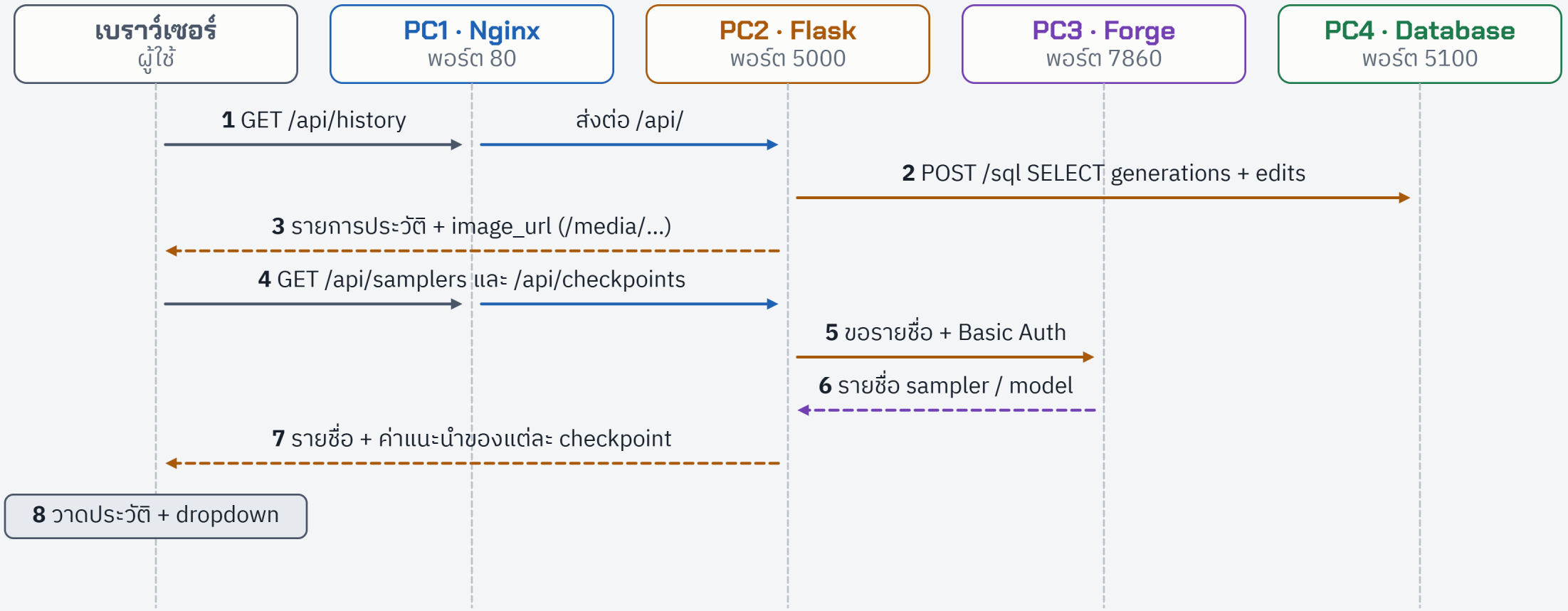
โหลดหน้าหลัก

หน้าเว็บขอข้อมูลจาก PC2 เอง 3 อย่าง: ประวัติ, รายชื่อ sampler และ checkpoint

ต่อไป: แต่ละคำขอวิ่งไปเครื่องไหน →

เข้าเว็บ · ขั้นตอนการทำงาน

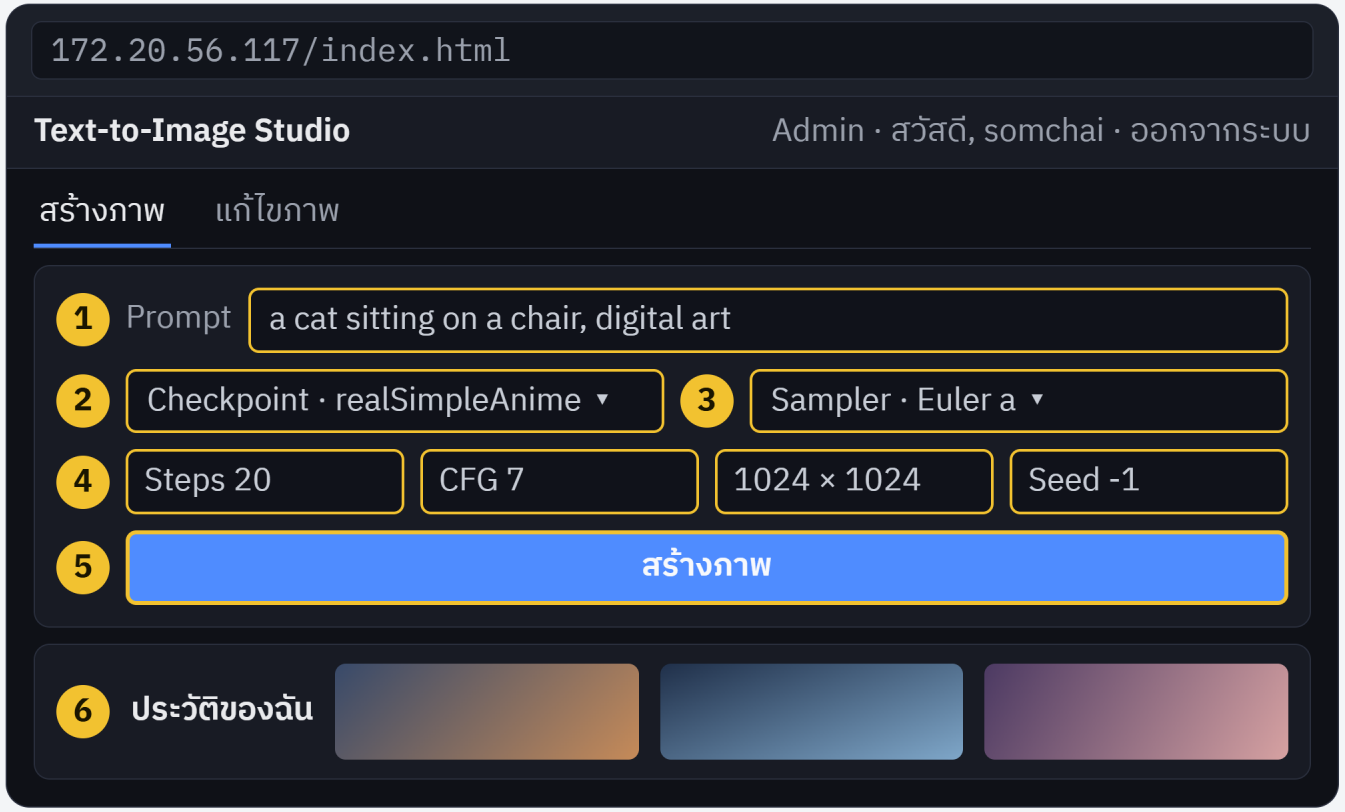
โหลดหน้าหลัก: ประวัติ และรายชื่อจาก AI



เบราว์เซอร์ไม่เคยคุยกับ PC3 ตรง ๆ · PC2 แนบรหัส API ไปถาม Forge แทน · โค้ด → ภาคผนวก C2, C8

สร้างภาพ · หน้าเว็บ

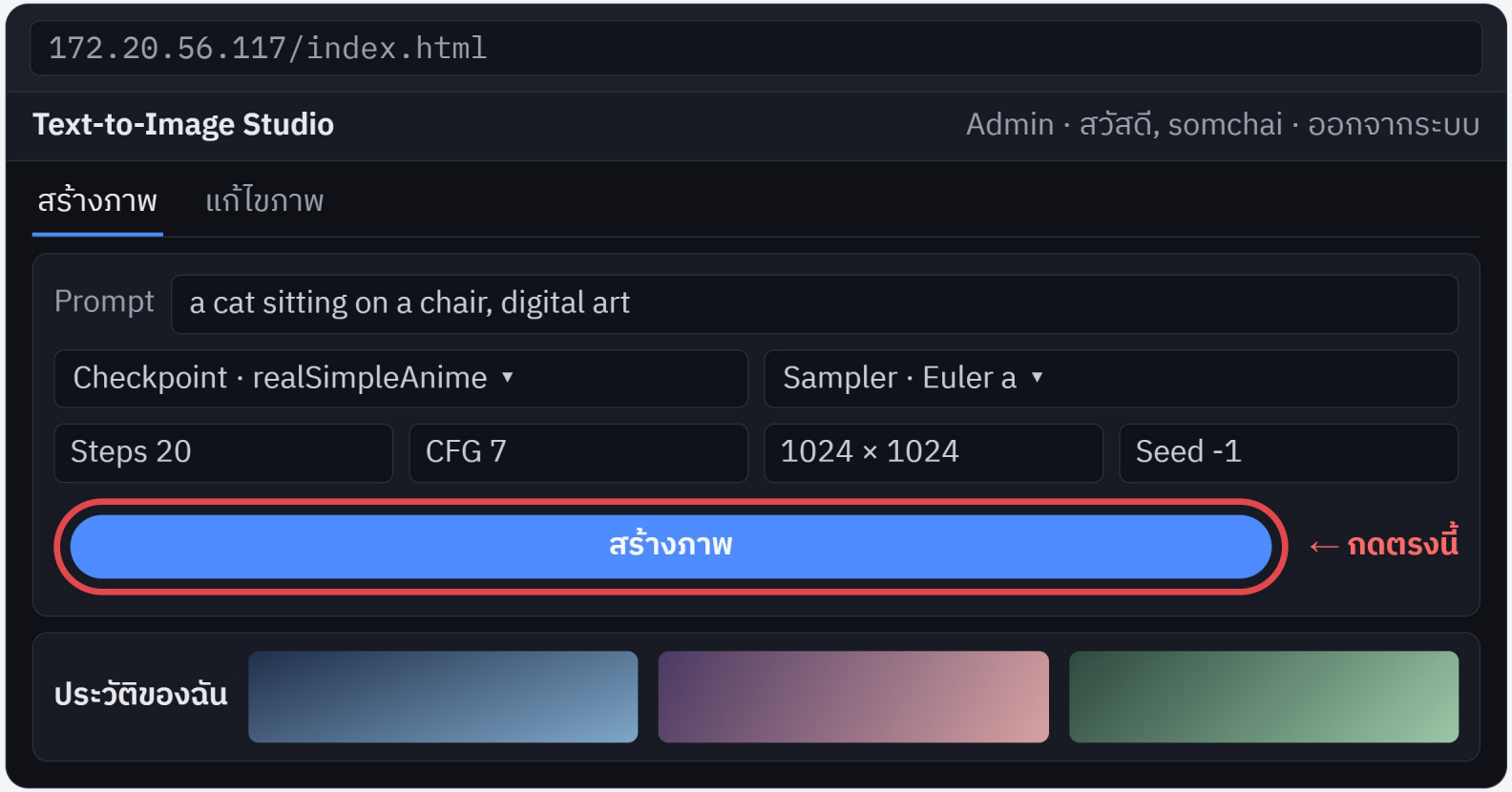
หน้าสร้างภาพ: แต่ละช่องทำอะไร



1. **Prompt**
   ข้อความที่อยากให้ AI วาด (มี negative prompt ด้วย)
2. **Checkpoint**
   โมเดลที่ใช้วาด รายชื่อมาจาก PC3 เลือกแล้วตั้งค่าแนะนำให้
3. **Sampler**
   วิธีลบนอยส์ทีละขั้น รายชื่อมาจาก PC3
4. **Steps · CFG · ขนาด · Seed**
   PC2 บีบค่าให้อยู่ในช่วงปลอดภัย (steps 1–50, ขนาด ≤ 1024)
5. **ปุ่มสร้างภาพ**
   รวมทุกช่องเป็น JSON แล้ว POST /api/generate
6. **ประวัติ**
   โหลดจาก /api/history ตัวภาพมาทาง /media/

สร้างภาพ · สิ่งที่ผู้ใช้ทำ

ผู้ใช้กด “สร้างภาพ”



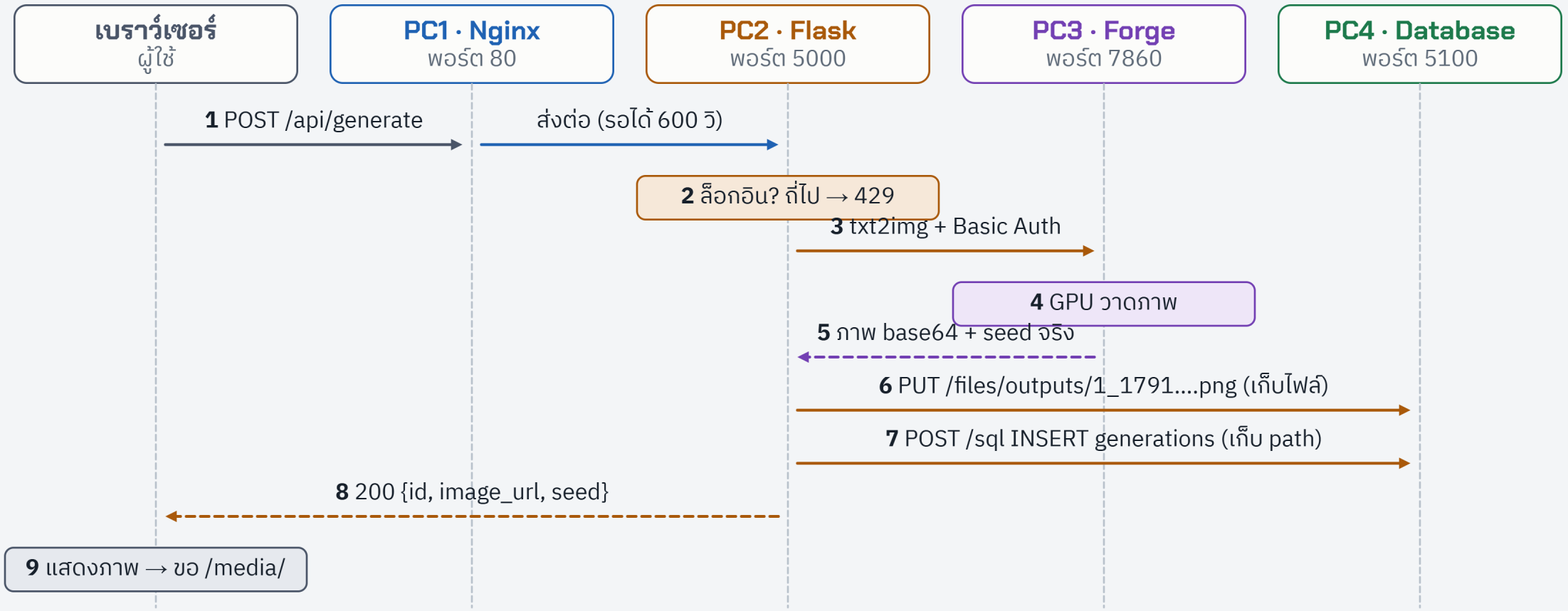
สร้างภาพ

รวมค่าทุกช่องเป็น JSON แล้ว POST /api/generate ระหว่างรอ ปุ่มเปลี่ยนเป็น “กำลังสร้างภาพ...”

ต่อไป: คำขอนี้วิ่งผ่านเครื่องไหนบ้าง →

สร้างภาพ · ขั้นตอนการทำงาน

กด “สร้างภาพ”



โค้ดของขั้นนี้ → ภาคผนวก C6 (generate) · C7 (เก็บภาพ) · C8, C9 (คุยกับ PC4)

สร้างภาพ · ข้อมูลที่ส่ง

แพ็กเก็ตที่วิ่งตอนสร้างภาพ

เบราว์เซอร์ → PC2 **POST /api/generate**

```
Cookie: session=…
{"prompt": "a cat sitting on a chair",
 "steps": 20, "cfg_scale": 7,
 "width": 1024, "height": 1024,
 "sampler": "Euler a", "seed": -1,
 "checkpoint": "realSimpleAnime_v10"}
```

PC2 → PC3 **POST /sdapi/v1/txt2img**

```
Authorization: Basic <รหัสจาก .env>
{"prompt": "a cat sitting on a chair",
 "steps": 20, "sampler_name": "Euler a",
 "override_settings":
   {"sd_model_checkpoint": "realSimple…"},
 "do_not_save_samples": true, …}
```

PC3 → PC2 **ผลลัพธ์จาก AI**

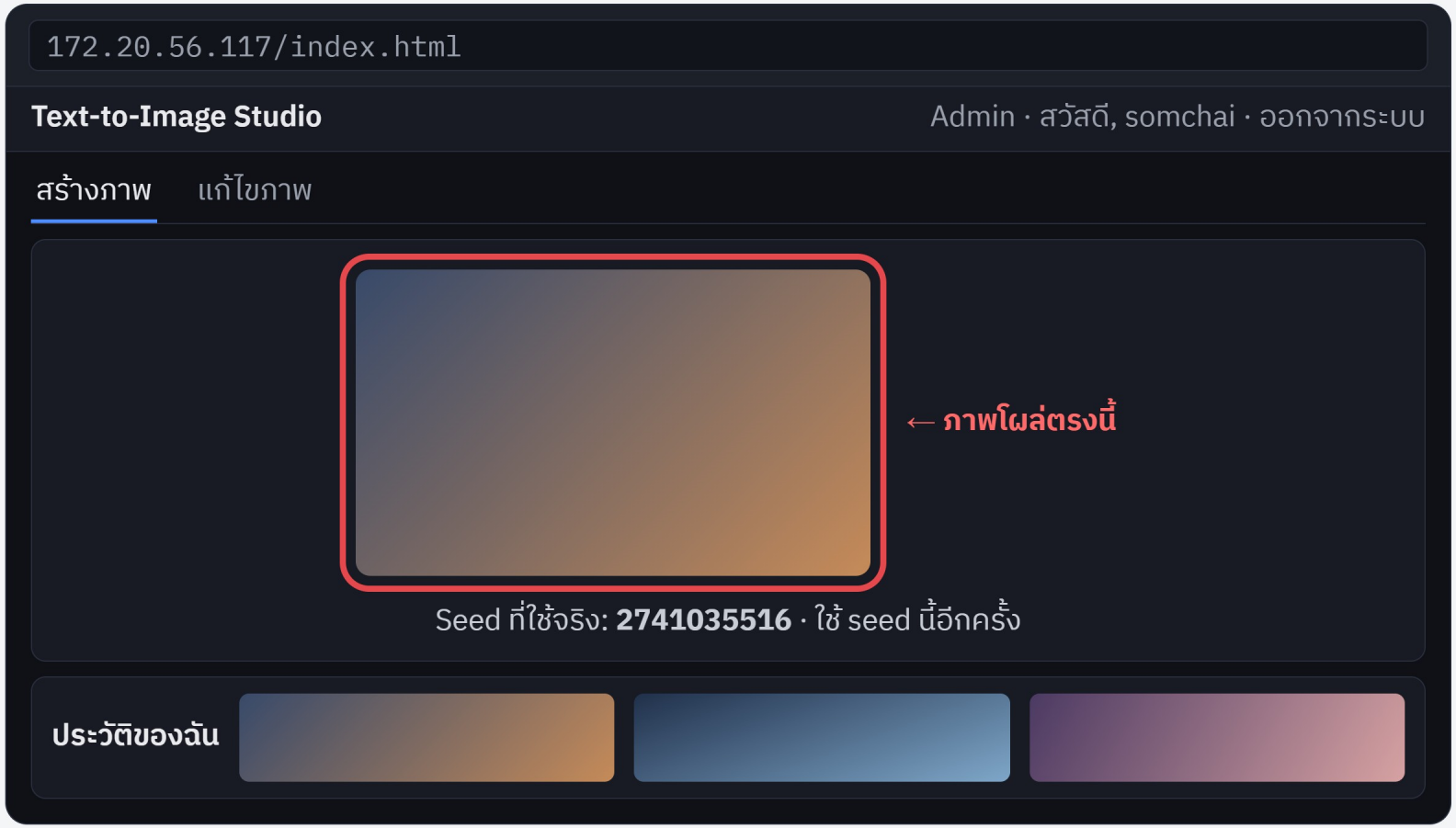
```
200 OK
{"images": ["iVBORw0KGgoAAAANSUhEUg…"],
 "info": "{\"seed\": 2741035516, …}"}
ภาพ PNG ถูกแปลงเป็นข้อความ base64
PC2 ถอดกลับเป็นไฟล์ แล้วส่งไปเก็บที่ PC4
```

PC2 → เบราว์เซอร์ **คำตอบสุดท้าย**

```
200 OK
{"id": 12,
 "image_url": "/media/outputs/1_1791….png",
 "prompt": "a cat sitting on a chair",
 "seed": 2741035516}
```

สร้างภาพ · สิ่งที่ผู้ใช้เห็น

ภาพที่สร้างเสร็จโผล่บนหน้าเว็บ



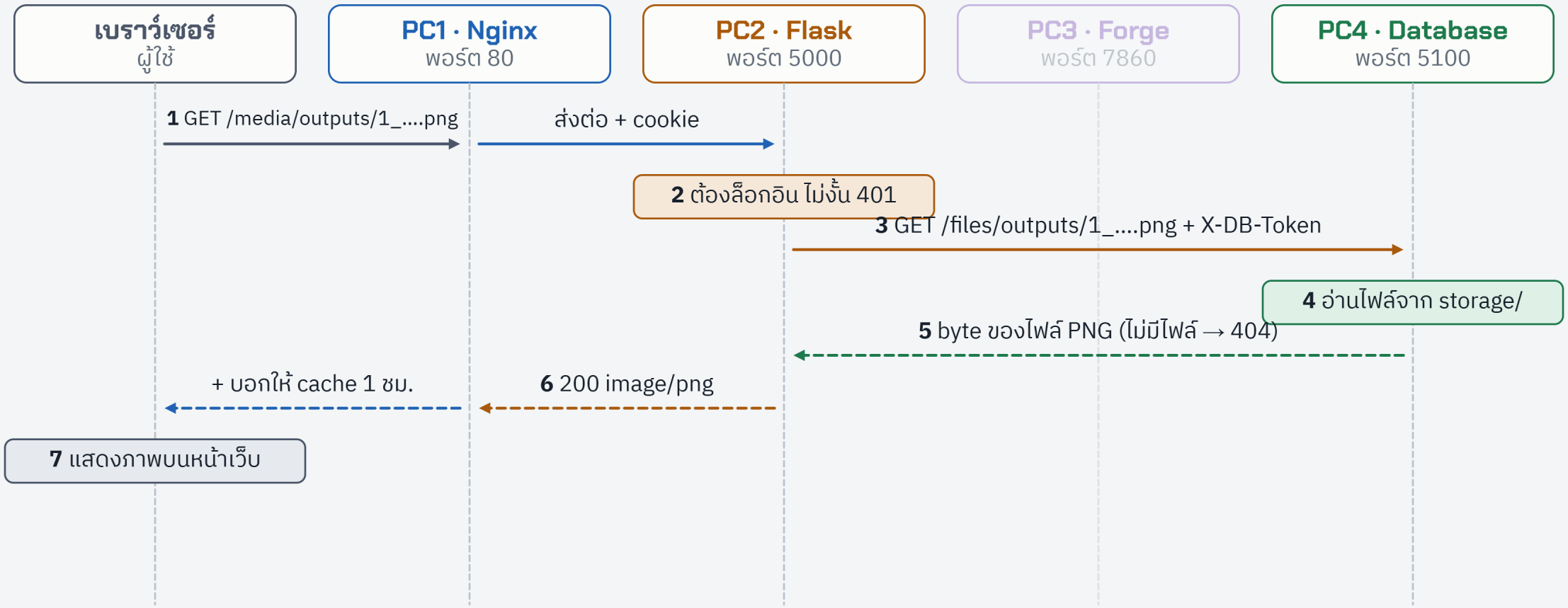
ภาพขึ้นบนจอ

หน้าเว็บได้มาแค่ URL /media/outputs/... แล้วเบราว์เซอร์ขอตัวภาพจาก PC1 เอง

ต่อไป: ตัวภาพเดินทางมาจากไหน →

สร้างภาพ · ขั้นตอนการทำงาน

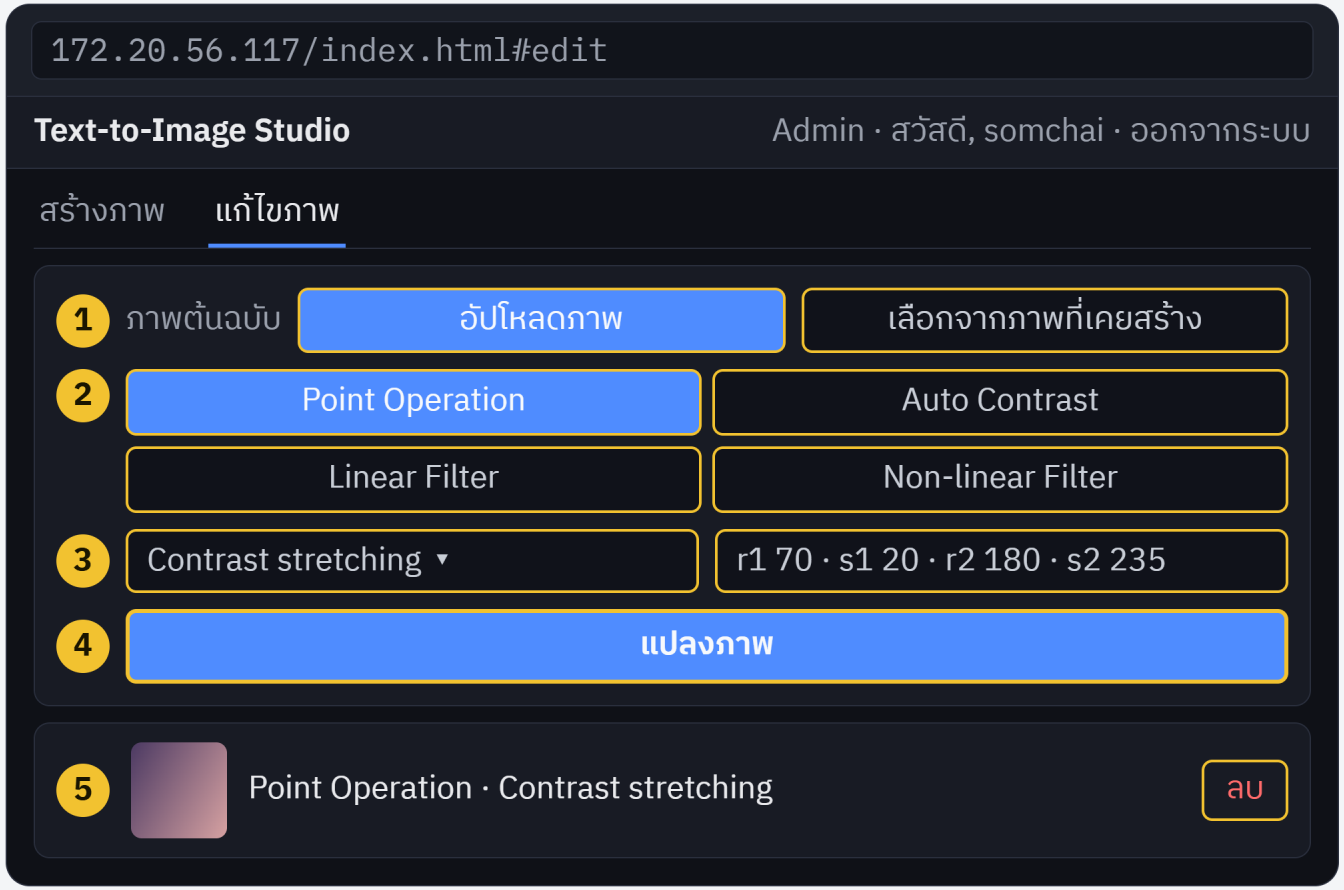
ภาพขึ้นบนจอได้ยังไง (/media/)



เบราว์เซอร์ไม่รู้ว่าภาพอยู่ที่ PC4 เห็นแค่ URL ของ PC1 · โค้ด → ภาคผนวก C1 (/media/) · C7 (media())

แก้ไขภาพ · หน้าเว็บ

หน้าแก้ไขภาพ: แต่ละช่องทำอะไร



1 แหล่งภาพ
อัปโหลดไฟล์ใหม่ หรือเลือกภาพเดิม (ส่งไปแค่ generation_id)

2 ฟังก์ชัน 4 กลุ่ม
ค่านี้คือ operation ที่ส่งไปให้ PC2

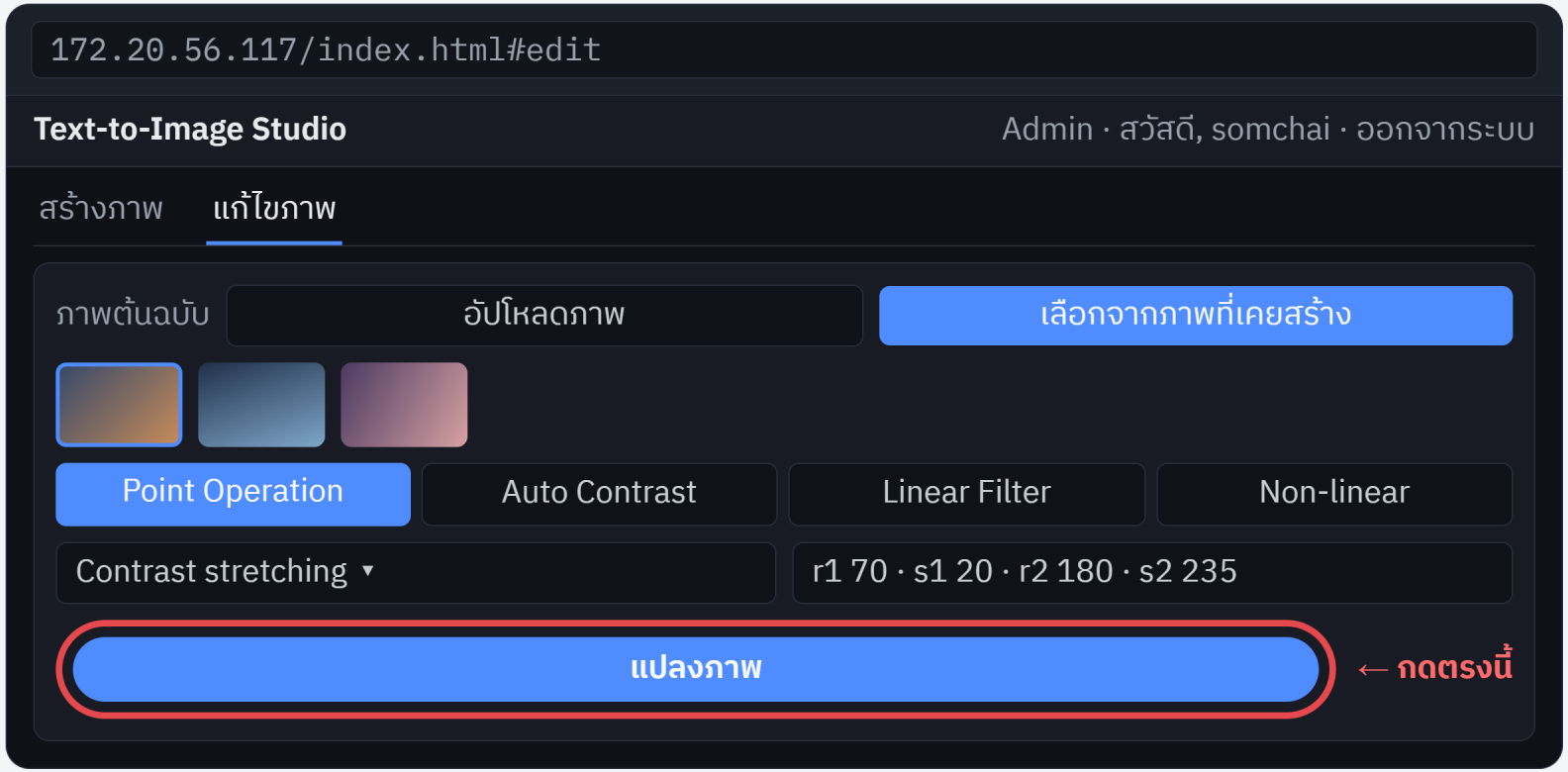
3 วิธีย่อย + ค่าตั้ง
โชว์เฉพาะช่องของวิธีที่เลือก ส่งไปเป็น method และค่าต่าง ๆ

4 ปุ่มแปลงภาพ
รวมเป็น FormData แล้ว POST /api/edit

5 ลบประวัติการแก้ไข
POST /api/edit/delete/<id> ลบได้เฉพาะของตัวเอง

แก้ไขภาพ · สิ่งที่ผู้ใช้ทำ

ผู้ใช้กด “แปลงภาพ”



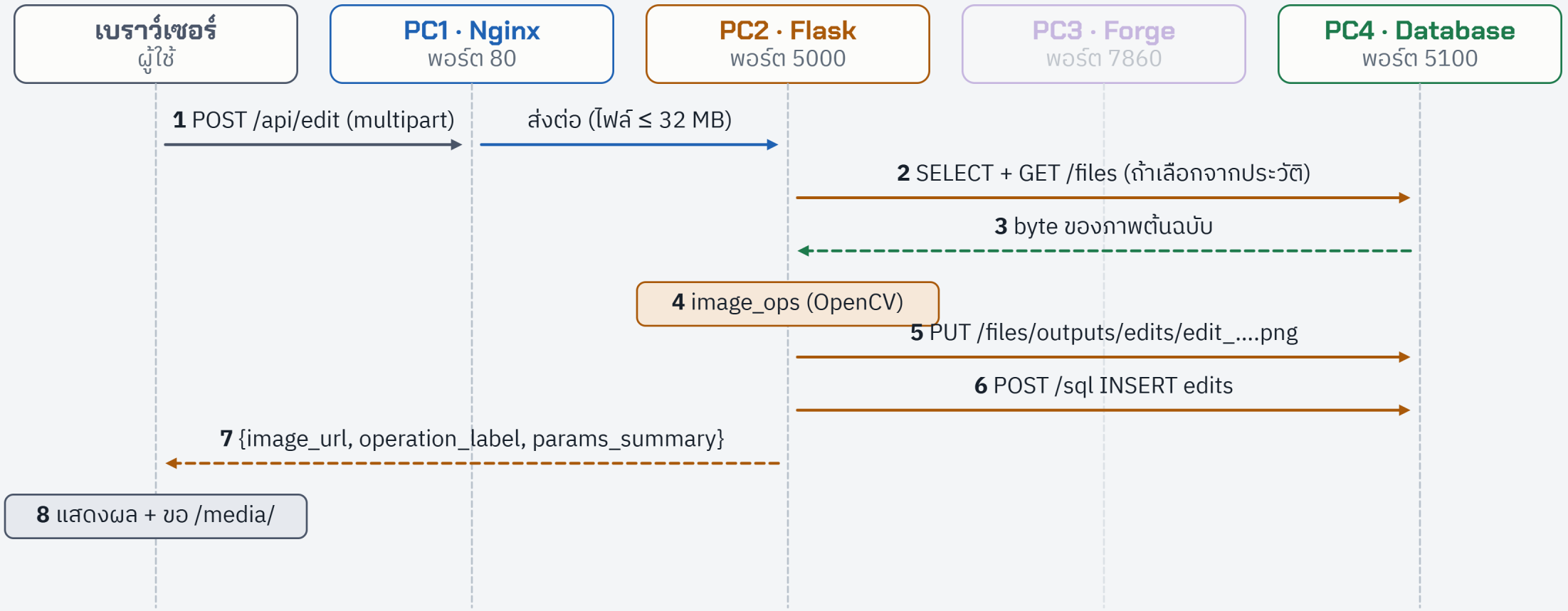
แปลงภาพ

รวมแหล่งภาพ ฟังก์ชัน วิธี และค่าที่ตั้ง เป็น FormData แล้ว POST /api/edit

ต่อไป: คำขอนี้วิ่งผ่านเครื่องไหนบ้าง →

แก้ไขภาพ · ขั้นตอนการทำงาน

กด “แปลงภาพ”



ถ้าอัปโหลดไฟล์เอง ข้ามขั้น 2–3 เพราะภาพมากับคำขอเลย · โค้ด → ภาคผนวก C10

แก้ไขภาพ · ใน PC2

image_ops.py: 4 กลุ่มที่ PC2 ประมวลผล

กลุ่ม (operation)	วิธี (method)	หลักการ	OpenCV ที่ใช้
Point Operation ·point_op	Negative · Log · Contrast stretching	แปลงค่าทีละพิกเซล ผ่านตาราง LUT	cv2.LUT
Automatic contrast ·auto_contrast	Auto contrast · Histogram Equalization	ยืด histogram หรือใช้ CDF ทำบนช่อง Y	cv2.LUT บนช่อง Y
Spatial filter (linear) ·linear_filter	Mean · Gaussian · Laplacian	ผลรวมถ่วงน้ำหนักของพิกเซลรอบ ๆ (kernel)	filter2D · GaussianBlur · Laplacian
Spatial filter (non-linear) ·nonlinear_filter	Median · Mean · Min · Max	เรียงค่าในหน้าต่าง แล้วเลือกตัวที่ต้องการ	medianBlur · blur · erode · dilate

> app.py เรียก apply_operation(img, operation, form) แล้วเลือก handler ของกลุ่มจาก _HANDLERS ค่าผิดตอบ 400

แก้ไขภาพ · ทดลองกับภาพจริง

กลุ่ม 1 · Point Operation

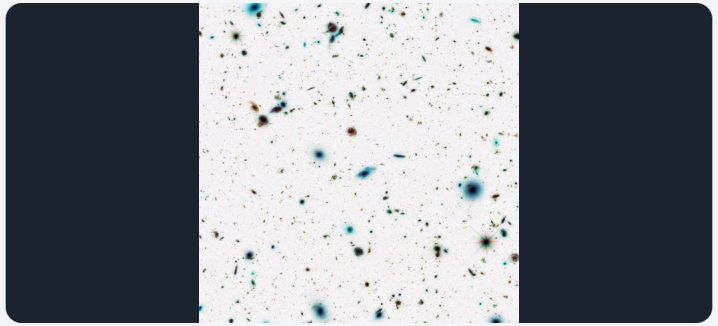
Negative

พื้นดำกลายเป็นพื้นขาว เห็นจุดเล็กชัดขึ้น

ก่อน



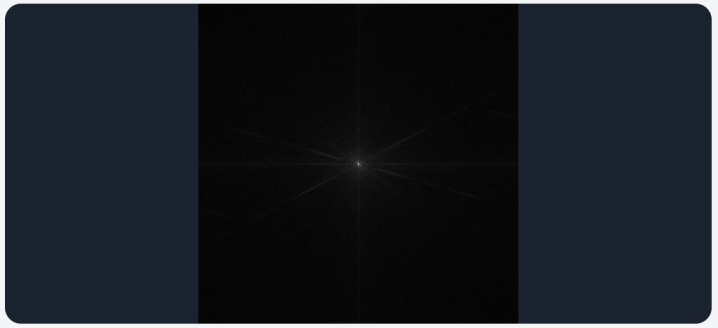
หลัง



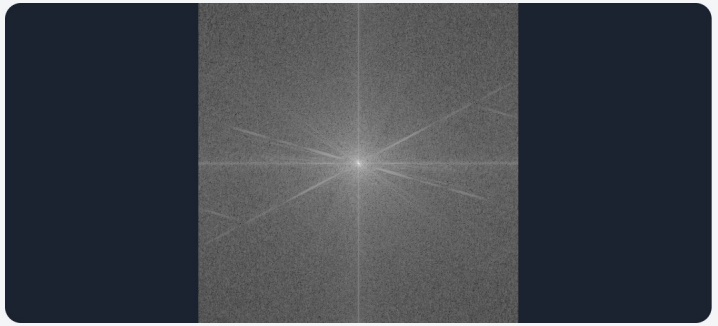
Log

spectrum ที่เกือบดำ กลายเป็นเห็นลวดลาย

ก่อน



หลัง



Contrast stretching

r1 90 · s1 20 · r2 160 · s2 235

ก่อน



หลัง



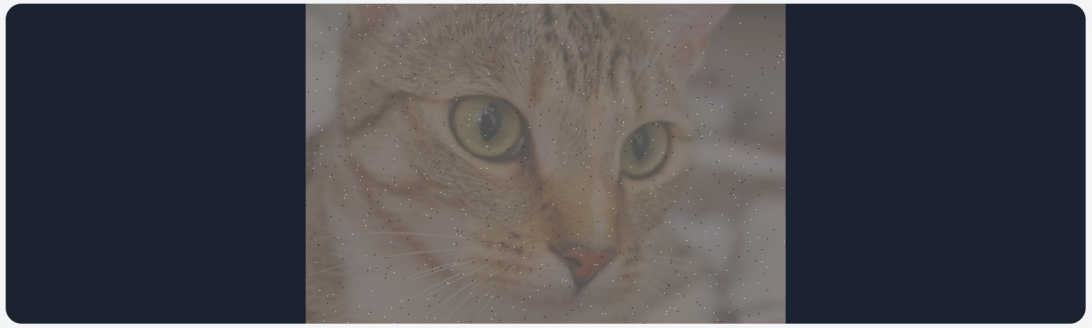
แก้ไขภาพ · ทดลองกับภาพจริง

กลุ่ม 2 · Automatic contrast

Automatic contrast adjustment

ตัดหัวท้าย 1% · ยืดช่วง 87–130 ให้เต็ม 0–255

ก่อน



หลัง



Histogram Equalization

ภาพมืด ค่ากองอยู่ช่วงต่ำ → กระจายเต็มช่วง

ก่อน



หลัง



แก้ไขภาพ · ทดลองกับภาพจริง

กลุ่ม 3 · Spatial filter แบบ linear

Mean 5×5

noise ลดลง แต่ภาพเบลอทั้งภาพ

ก่อน



หลัง



Gaussian 5×5 · sigma 1.5

ภาพเดียวกัน เบลอนุ่มกว่า Mean

ก่อน



หลัง



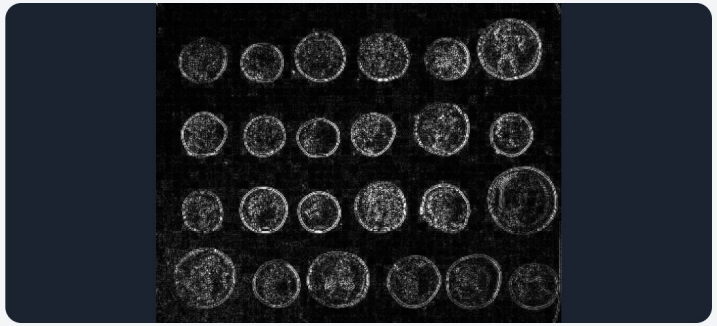
Laplacian 8 ทิศ

พื้นเรียบหายไป เหลือแต่ขอบ

ก่อน



หลัง



แก้ไขภาพ · ทดลองกับภาพจริง

กลุ่ม 4 · Spatial filter แบบ non-linear

Median 3×3

จุดขาวดำหายเกือบหมด

ก่อน



หลัง



Mean 3×3

ภาพเดียวกัน จุดกลายเป็นรอยจาง

ก่อน



หลัง



Min 3×3

ลบจุดขาว (salt)

ก่อน



หลัง



Max 3×3

ลบจุดดำ (pepper)

ก่อน



หลัง



แก้ไขภาพ · ปุ่มปรับค่า

ปุ่มปรับค่าแต่ละตัว ปรับอะไร

วิธี	ปุ่มปรับ (ช่วงค่า)	ปรับอะไร	ค่ามากขึ้น → ผลที่ได้
Contrast stretching	r1, s1, r2, s2 (r 1–254, s 0–255)	จุดหักของเส้นที่แปลงค่าเดิม r เป็นค่าใหม่ s	ช่วง r1–r2 แคบลง หรือ s1–s2 ห่างขึ้น → contrast สูงขึ้น (r1 ต้องน้อยกว่า r2)
Automatic contrast	ตัดหัวท้าย % (0–5)	ตัดค่ามืดสุด/สว่างสุดทิ้งฝั่งละกี่ % ก่อนยืด	ภาพเข้มขึ้น แต่ส่วนมืดสุด/สว่างสุดกลายเป็นดำ/ขาวล้วน
Mean · Gaussian	ขนาด kernel (3×3 – 15×15)	จำนวนพิกเซลรอบ ๆ ที่เอามาคิด	noise หายมากขึ้น แต่ภาพเบลอมากขึ้น
Gaussian	sigma (0.5 – 10)	น้ำหนักกระจายออกไปรอบนอกแค่ไหน	เบลอมากขึ้น (ไม่เกินขนาด kernel)
Laplacian	4 ทิศ / 8 ทิศ	ดูแค่บน ล่าง ซ้าย ขวา หรือรวมแนวทแยงด้วย	8 ทิศ: ขอบชัดและหนาขึ้น จับขอบเฉียงได้ดีกว่า
Median · Mean · Min · Max	ขนาดหน้าต่าง (3×3 – 9×9)	จำนวนพิกเซลรอบ ๆ ที่เอามาเรียง	ลบ noise ก้อนใหญ่ได้ แต่รายละเอียดหาย · Min/Max ขยายส่วนมืด/สว่างเป็นบล็อก

Negative · Log · Histogram Equalization ไม่มีค่าให้ปรับ (Log คำนวณ c = 255 / log 256 ให้เอง)

แก้ไขภาพ · ปุ่มปรับค่า

Contrast stretching: ปรับ r1, s1, r2, s2

ภาพต้นฉบับ

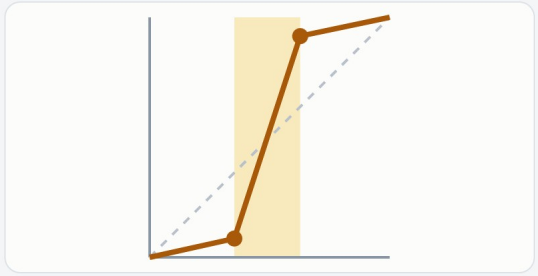
ค่าทั้งภาพอยู่แค่ 90–160 ภาพจึงจืด

แกนนอน r = ค่าเดิม
แกนตั้ง s = ค่าใหม่
เส้นส้ม = ตารางแปลงค่า (LUT)
แถบเหลือง = ช่วง r1–r2
เส้นประ = s เท่ากับ r



เส้นชัน = ยืด

ช่วง 90–160 ถูกยืดเป็น 20–235 สีสดขึ้นมาก

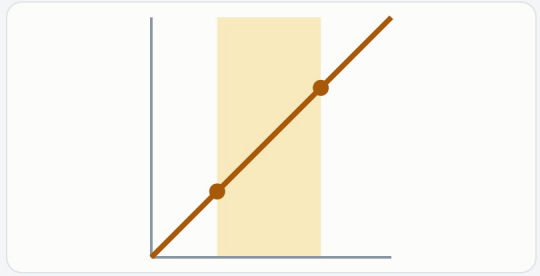


```
(r1, s1) = (90, 20)
(r2, s2) = (160, 235)
```



เส้นทแยง 45° = เหมือนเดิม

ตั้ง s เท่ากับ r ทุกจุด ค่าไม่เปลี่ยน

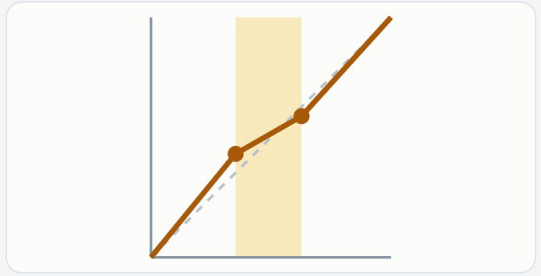


```
(r1, s1) = (70, 70)
(r2, s2) = (180, 180)
```



เส้นแบน = บีบ

ช่วง 90–160 ถูกบีบเหลือ 110–150 ภาพจืดลงอีก



```
(r1, s1) = (90, 110)
(r2, s2) = (160, 150)
```

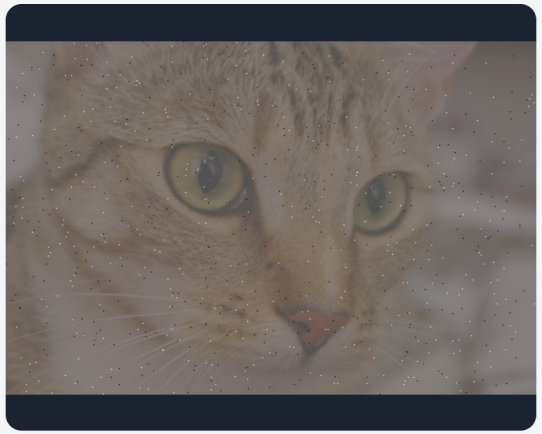


แก้ไขภาพ · ปุ่มปรับค่า

Automatic contrast: ปรับ “ตัดหัวท้าย %”

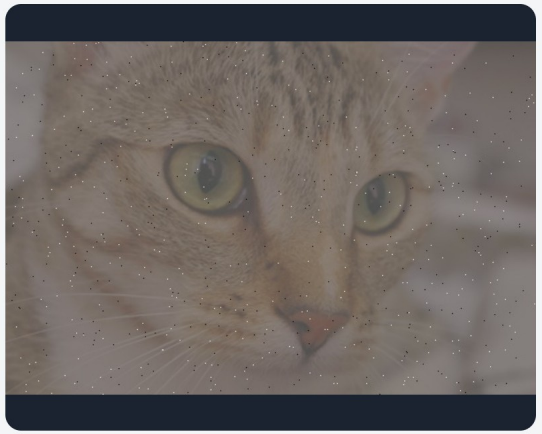
ภาพต้นฉบับ

ภาพจืด มีจุดขาวดำแทรกอยู่ไม่กี่จุด



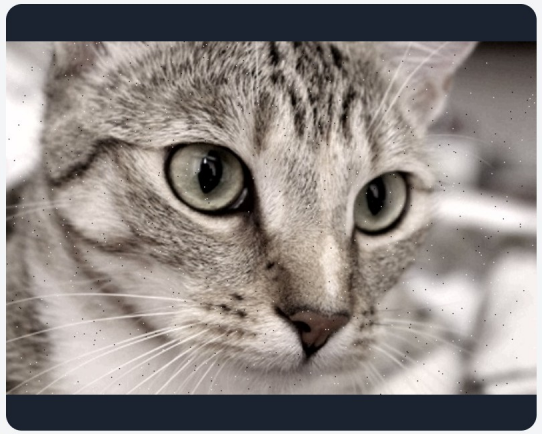
0%

เจอค่า 0 และ 255 จากจุดขาวดำ ยืดไม่ได้ ภาพแทบไม่เปลี่ยน



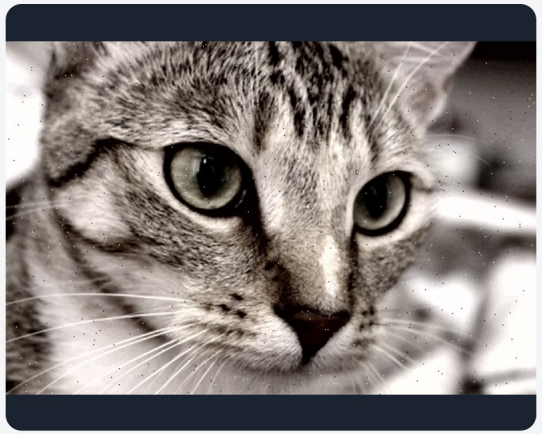
1% (ค่าเริ่มต้น)

ตัดจุดขาวดำทิ้ง ยืดช่วง 87–130 ให้เต็ม 0–255



5%

ยืดช่วงแคบลงเหลือ 97–126 เข้มกว่า แต่ส่วนสว่างกลายเป็นขาวล้วน



> ค่ามากขึ้น = ช่วงที่ยืดแคบลง ภาพเข้มขึ้น แต่ส่วนมืดสุด/สว่างสุดถูกตัดเป็นดำ/ขาวล้วนมากขึ้น

แก้ไขภาพ · ปุ่มปรับค่า

Linear filter: ขนาด kernel, sigma, 4/8 ทิศ

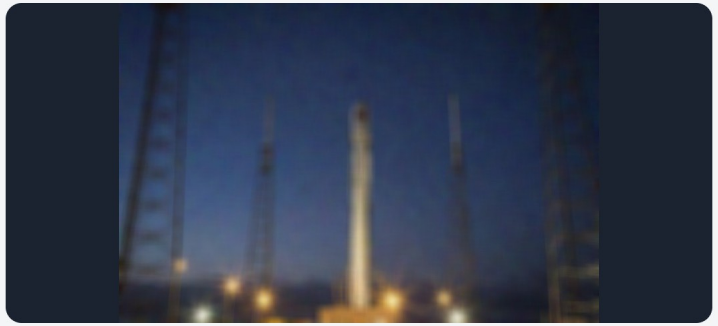
Mean · ขนาด kernel

ใหญ่ขึ้น noise หาย แต่ภาพเบลอมาก

3 × 3



15 × 15



Gaussian · sigma (kernel 15 × 15)

มากขึ้น น้ำหนักกระจายออกรอบนอก เบลอขึ้น

sigma 0.5



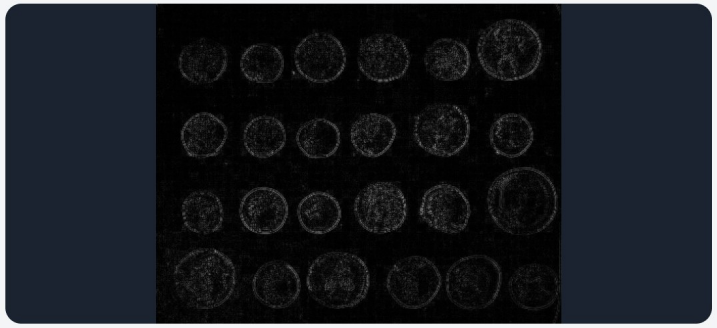
sigma 5



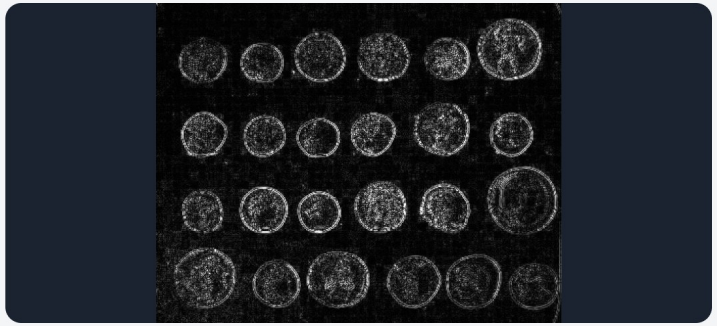
Laplacian · 4 ทิศ / 8 ทิศ

8 ทิศดูแนวทแยงด้วย ขอบชัดและหนากว่า

4 ทิศ



8 ทิศ



แก้ไขภาพ · ปุ่มปรับค่า

Non-linear filter: ขนาดหน้าต่าง

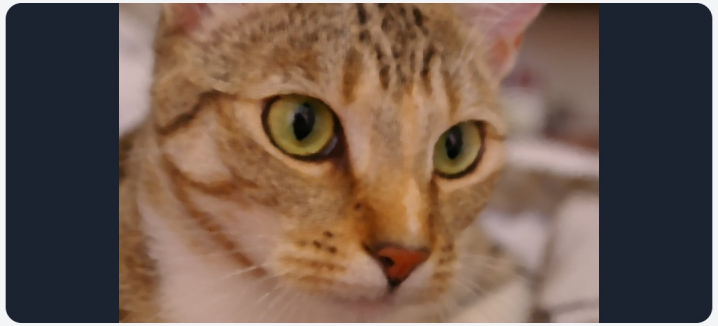
Median · noise หนาแน่น 30%

หน้าต่างใหญ่ขึ้น ลบ noise หนัก ๆ ได้ แต่ภาพนุ่มลง

3 × 3



7 × 7



Min · จุดขาว

หน้าต่างใหญ่ขึ้น ส่วนมืดขยายเป็นบล็อก

3 × 3



9 × 9



Max · จุดดำ

หน้าต่างใหญ่ขึ้น ส่วนสว่างขยายเป็นบล็อก

3 × 3

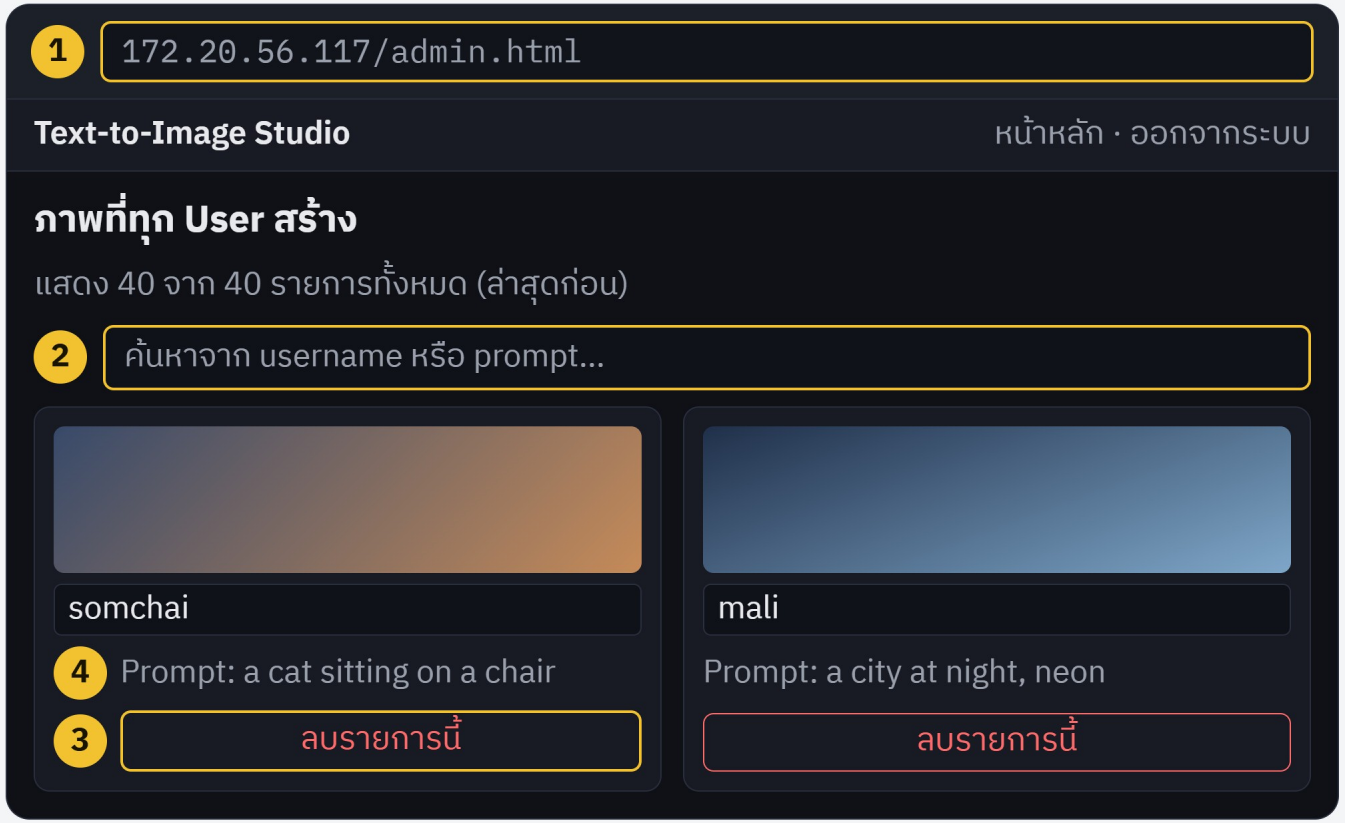


9 × 9



Admin · หน้าเว็บ

หน้า Admin: เห็นภาพของทุกคน



1 เปิดหน้า admin.html
GET /api/admin/records · PC2 เช็ค is_admin ซ้ำ ไม่ใช่ admin ได้ 403

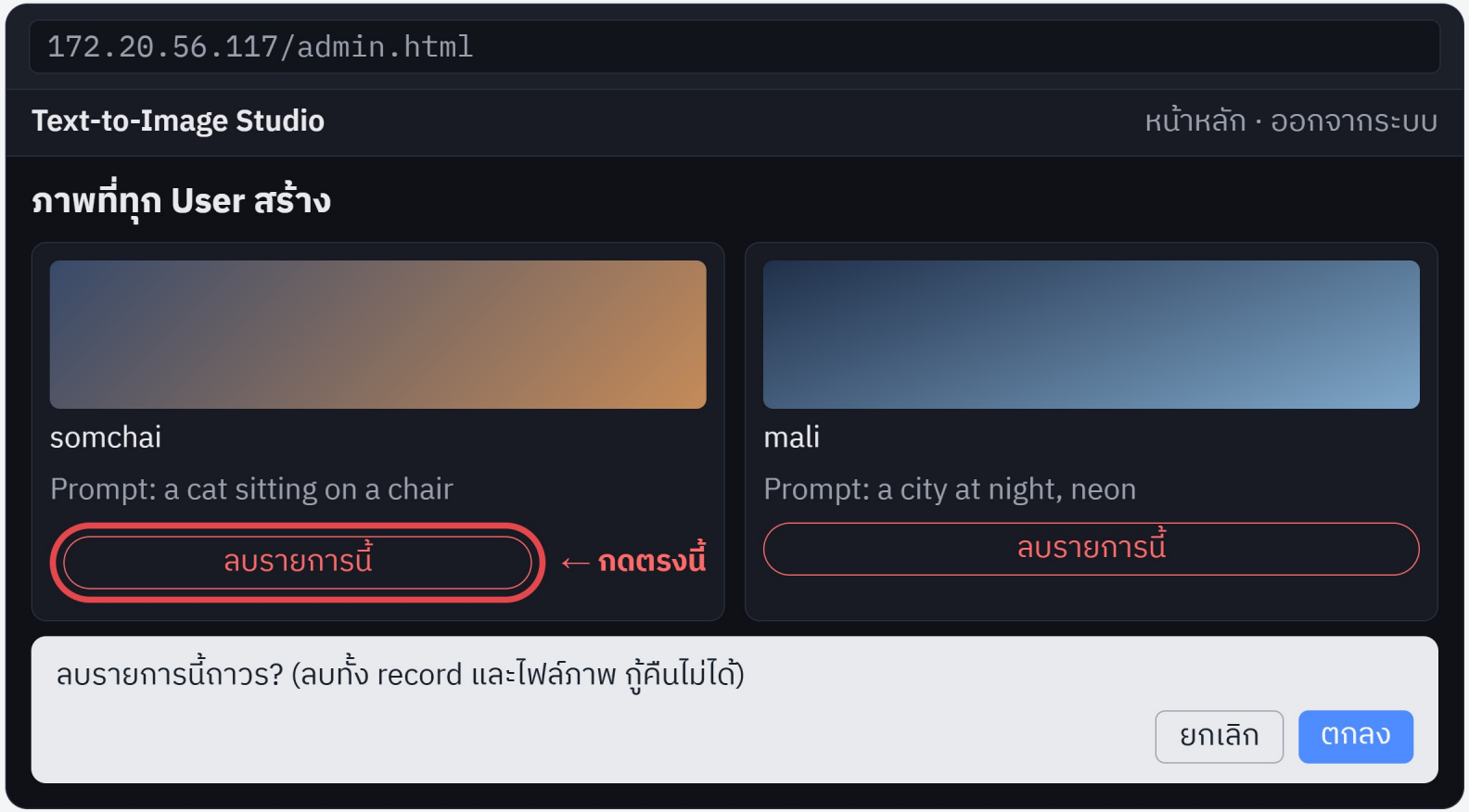
2 ช่องค้นหา
กรองการ์ดในเบราว์เซอร์อย่างเดียว ไม่ส่งอะไรไป PC2

3 ลบรายการนี้
POST /api/admin/delete/<id> ดูลำดับขั้นในสไลด์ถัดไป

4 prompt ของคนอื่น
ผ่าน escapeHtml() ก่อนแสดง กันโค้ดแปลกปลอมรันในเบราว์เซอร์ admin (XSS)

Admin · สิ่งที่ผู้ใช้ทำ

Admin กด “ลบรายการนี้”



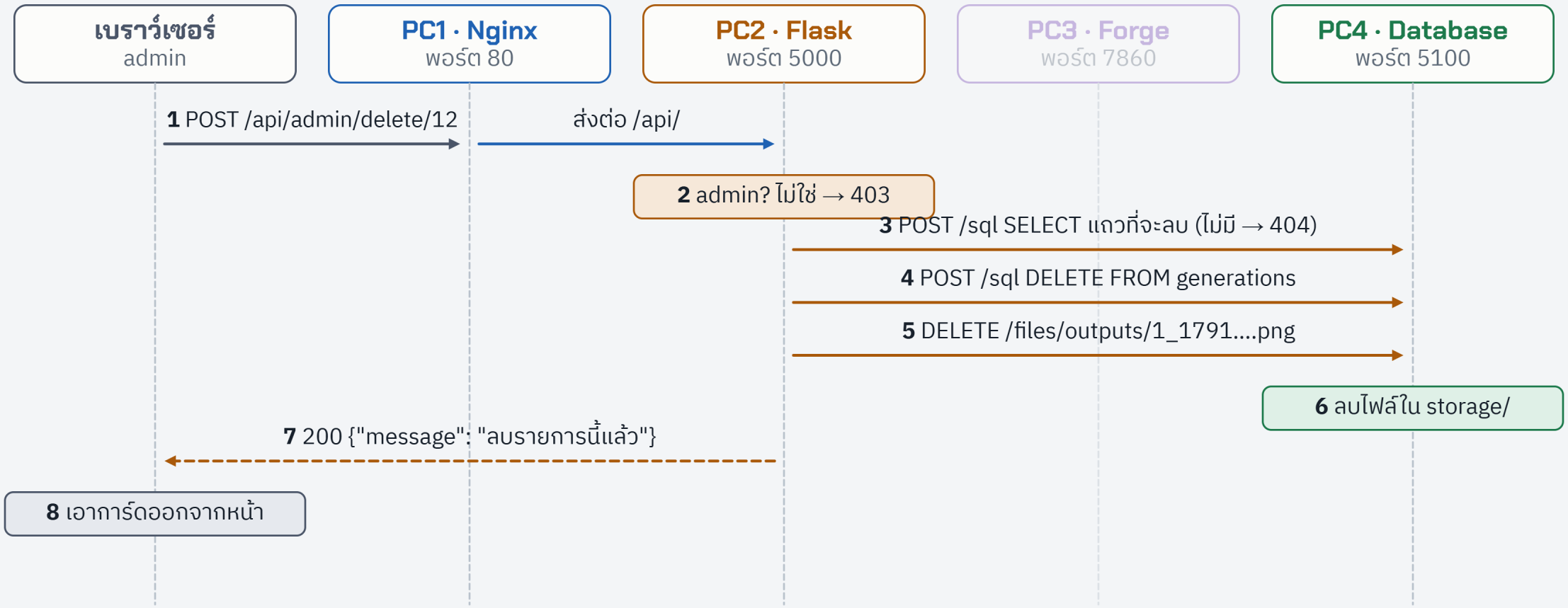
ลบรายการนี้

ยืนยันในกล่อง confirm แล้วส่ง POST /api/admin/delete/12

ต่อไป: คำขอนี้วิ่งผ่านเครื่องไหนบ้าง →

Admin · ขั้นตอนการทำงาน

กด “ลบรายการนี้”



ลบ record ก่อนแล้วค่อยลบไฟล์ · ผู้ใช้ลบประวัติแก้ไขของตัวเองใช้ลำดับเดียวกัน · โค้ด → ภาคผนวก C11

อื่น ๆ · ขั้นตอนการทำงาน

ออกจากระบบ และตรวจสถานะทุกเครื่อง

กด “ออกจากระบบ”

1. POST /api/logout ผ่าน PC1 ไป PC2
2. PC2 เรียก logout_user() ล้าง session
3. ตอบ Set-Cookie ที่หมดอายุแล้ว cookie เดิมใช้ไม่ได้อีก
4. เบราว์เซอร์ย้ายไป login.html

ต่อ PC2 ไม่ติด ก็ยังพากลับหน้า login · โค้ดอยู่ใน common.js และ app.py

เปิด /api/health

1. เปิด 172.20.56.117/api/health ไม่ต้องล็อกอิน
2. PC2 ลองต่อ PC3 และ PC4
3. ตอบสถานะทุกเส้นเป็น JSON

```
{"backend": {"status": "ok"},
 "ai_server_pc3": {"status": "ok", "models": 8},
 "database_pc4": {"status": "ok", "users": 3}}
```

เส้นไหนขาด จะขึ้น error พร้อมรายละเอียดตรงนั้น

การเชื่อมต่อ

4 เส้นทาง และค่าที่ต้องตรงกันสองฝั่ง

เส้นทาง	พอร์ต	ยืนยันตัวตน	ฝั่งผู้ส่ง ต้องตั้ง	ฝั่งผู้รับ ต้องตั้ง
เบราว์เซอร์ → PC1	80	cookie session	ผู้ใช้พิมพ์ http://172.20.56.117	imageproject.conf · listen 80
PC1 → PC2	5000	ส่ง cookie ของผู้ใช้ต่อ	imageproject.conf · upstream 172.20.56.107:5000	app.py · app.run(host="0.0.0.0", port=5000) · Firewall เปิด 5000
PC2 → PC3	7860	Basic Auth	config.py · FORGE_API_URL · .env · FORGE_API_USER / PASS	Forge · --api --api-auth user:pass --listen · Firewall รับแค่ PC2
PC2 → PC4	5100	X-DB-Token + IP	config.py · DB_SERVER_URL · .env · DB_TOKEN	.env · DB_TOKEN ค่าเดียวกัน · ALLOWED_CLIENT_IPS = IP ของ PC2

API ของแต่ละเครื่อง

แต่ละเครื่องเปิด API อะไร ใครเป็นคนเรียก

PC1 · Nginx

พอร์ต 80 · เบราว์เซอร์เรียก

`/`
ส่งไฟล์หน้าเว็บ

`/api/*`
ส่งต่อไป PC2

`/media/*`
ส่งต่อไป PC2

ไม่มีรหัส ส่ง cookie ต่อให้ PC2

PC2 · Flask

พอร์ต 5000 · เบราว์เซอร์เรียกผ่าน PC1

`/api/register` · `/api/login`
`/api/logout` · `/api/me`
`/api/history`
`/api/samplers`
`/api/checkpoints`
`/api/generate` · `/media/…`
`/api/edit` · `/api/edit/delete`
`/api/admin/records`
`/api/admin/delete`
`/api/health`

ยืนยันตัวตนด้วย cookie session

PC3 · Forge

พอร์ต 7860 · PC2 เรียกเท่านั้น

`/sdapi/v1/samplers`
รายชื่อ sampler

`/sdapi/v1/sd-models`
รายชื่อ checkpoint

`/sdapi/v1/txt2img`
สั่งวาดภาพ

Basic Auth + Firewall รับแค่ PC2

PC4 · Database

พอร์ต 5100 · PC2 เรียกเท่านั้น

`/sql` · `/sql/batch`
อ่าน/เขียนฐานข้อมูล

`/files/<path>`
เก็บ · อ่าน · ลบไฟล์ภาพ

`/health`
นับข้อมูล เช็คว่ายังทำงาน

X-DB-Token + IP ของ PC2

API ของแต่ละเครื่อง · PC2 (1/2)

PC2 · บัญชีผู้ใช้ และหน้าหลัก

Method	Path	ส่งไป	ได้กลับ	PC2 คุยต่อกับ
POST	/api/register	{username, password, confirm}	201 {message} · 400 · 409	PC4
POST	/api/login	{username, password}	200 {user} + Set-Cookie · 401	PC4
POST	/api/logout	cookie	200 {message} + ลบ cookie	–
GET	/api/me	cookie	200 {user} · 401	PC4
GET	/api/history	cookie	{generations, edits}	PC4
GET	/api/samplers	cookie	["Euler a", ...]	PC3
GET	/api/checkpoints	cookie	[{title, model_name, sampler, ...}]	PC3

ทุก API ตอบเป็น JSON · ผิดพลาดตอบ {"error": "ข้อความ"} พร้อม status 4xx/5xx · cookie ถูกแนบไปเองทุกคำขอหลังล็อกอิน

API ของแต่ละเครื่อง · PC2 (2/2)

PC2 · ภาพ Admin และสถานะ

Method	Path	ส่งไป	ได้กลับ	PC2 คุยต่อกับ
POST	/api/generate	{prompt, steps, cfg_scale, sampler, seed, ...}	{id, image_url, seed} · 429 · 502	PC3 + PC4
GET	/media/<path>	cookie	ไฟล์ PNG · 401 · 404	PC4
POST	/api/edit	FormData: operation, method, ภาพ หรือ id	{image_url, operation_label, ...} · 400	PC4
POST	/api/edit/delete/<id>	cookie	{message} · 404	PC4
GET	/api/admin/records	cookie ของ admin	{records, total_count} · 403	PC4
POST	/api/admin/delete/<id>	cookie ของ admin	{message} · 403 · 404	PC4
GET	/api/health	– ไม่ต้องล็อกอิน	สถานะของ PC2, PC3, PC4	PC3 + PC4

429 = กดสร้างภาพถี่เกินไป · 502 = ต่อ PC3 ไม่ได้ · 503 = ต่อ PC4 ไม่ได้ · 403 = ไม่ใช่ admin

API ของแต่ละเครื่อง · PC3

PC3 · Forge API ที่ PC2 เรียก

ทุกคำขอแนบ **Authorization: Basic <user:pass จาก .env>** · Firewall ให้พอร์ต 7860 รับแค่ PC2 (172.20.56.107)

Method	Path	ส่งไป	ได้กลับ	PC2 เรียกตอน
GET	/sdapi/v1/samplers	–	[{name: "Euler a"}, ...]	/api/samplers
GET	/sdapi/v1/sd-models	–	[{title, model_name}, ...]	/api/checkpoints · /api/health
POST	/sdapi/v1/txt2img	{prompt, steps, sampler_name, seed, override_settings, ...}	{images: [base64], info}	/api/generate

รหัสไม่ตรงกับ --api-auth → Forge ตอบ 401 → dropdown ว่าง สร้างภาพไม่ได้

txt2img ใช้ timeout (10, 300): ต่อไม่ติดใน 10 วิ = PC3 ดับ → 502

ภาพกลับมาเป็นข้อความ base64 เพราะใส่ไฟล์ใน JSON ตรง ๆ ไม่ได้

API ของแต่ละเครื่อง · PC4

PC4 · Database API ที่ PC2 เรียก

ทุกคำขอ (ยกเว้น /health) ต้องมี **X-DB-Token** ตรงกับ .env และมาจาก IP ของ PC2 ไม่งั้นได้ 401 / 403

Method	Path	ส่งไป	ได้กลับ	PC2 เรียกตอน
POST	/sql	{sql, params, fetch}	{rows, lastrowid, rowcount}	ทุกครั้งที่อ่าน/เขียนข้อมูล
POST	/sql/batch	{statements: [{sql, params}, ...]}	{lastrowids}	หลายคำสั่งในครั้งเดียว (มีไว้ ยังไม่ได้ใช้)
PUT	/files/outputs/...	byte ของไฟล์ PNG	{path, size}	เก็บภาพใหม่ (สร้าง / แปลงภาพ)
GET	/files/outputs/...	–	byte ของไฟล์ · 404	/media/ และแปลงภาพจากประวัติ
DELETE	/files/outputs/...	–	{deleted: true}	ลบรายการ
GET	/health	– ไม่ต้องใช้ token	{status, users, generations, edits}	/api/health

ความปลอดภัย

8 จุดที่ป้องกันไว้

รหัสผ่านเป็น hash

ไม่เก็บรหัสจริง ล็อกอินใช้การเทียบ hash

cookie ปลอมไม่ได้

เซ็นด้วย SECRET_KEY และเป็น HttpOnly

PC4 รับแค่ PC2

ต้องมี X-DB-Token และมาจาก IP ของ PC2

Forge ต้องใส่รหัส

--api-auth และ Firewall พอร์ต 7860 รับแค่ PC2

กดถี่ไม่ได้

สร้างภาพได้ 1 ครั้งต่อ 3 วินาที เกินได้ 429

กัน XSS

ทุกข้อความผ่าน escapeHtml() ก่อนใส่หน้าเว็บ

ภาพต้องล็อกอิน

/media/ มี @login_required คนนอกเปิดไม่ได้

รหัสอยู่ใน .env

แยกจากโค้ด และไม่ถูก commit ลง git

ถาม–ตอบ

C1 Nginx config
C2 apiJson / requireLogin
C3 auth.js preventDefault
C4 สมัคร / ล็อกอิน (hash)
C5 login_required, /api/me
C6 generate: 429 + เรียก AI
C7 เก็บภาพ + /media/
C8 db_client (SQL, ไฟล์)
C9 db_server (token, /sql)
C10 /api/edit + image_ops
C11 admin ลบรายการ
C12 config.py + .env

ภาคผนวก C1 · โค้ด

PC1 · Nginx ส่งต่อ /api/ และ /media/

pc1-frontend/nginx/imageproject.conf · บรรทัด 15–70 (ตัดมาเฉพาะส่วนสำคัญ)

```
upstream backend_pc2 {
    server 172.20.56.107:5000; # IP ของ PC2
}
server {
    listen 80; client_max_body_size 32M; # อัปโหลดภาพใหญ่ได้
    location / { # 1) หน้าเว็บ static บน PC1
        root "C:\image\ImageProject\pc1-frontend\html";
        try_files $uri $uri/ =404;
    }
    location /api/ { # 2) API → PC2
        proxy_pass http://backend_pc2;
        proxy_read_timeout 600s; # รอ GPU วาดภาพได้นาน
    }
    location /media/ { # 3) ไฟล์ภาพ → PC2
        proxy_pass http://backend_pc2;
        add_header Cache-Control "public, max-age=3600";
    }
}
```

ภาคผนวก C2 · โค้ด

เบราว์เซอร์ · requireLogin() เช็คล็อกอิน

pc1-frontend/html/js/common.js · บรรทัด 52–76

```
async function requireLogin({ adminOnly = false } = {}) {
  let data;
  try {
    data = await apiJson("/api/me"); // fetch + แนบ cookie + โยน error ถ้าไม่ใช่ 2xx
  } catch (err) { // 401 = ยังไม่ล็อกอิน · error อื่น = ต่อ PC2 ไม่ได้
    if (err.status === 401) {
      window.location.href = "login.html";
      return null;
    }
  }
  const user = data.user;
  if (adminOnly && !user.is_admin) { // หน้า admin แต่ไม่ใช่ admin
    window.location.href = "index.html";
    return null;
  }
  renderNavbar(user); // ชื่อผู้ใช้ + ลิงก์ Admin
  return user;
}
```

ภาคผนวก C3 · โค้ด

เบราว์เซอร์ · auth.js ส่งฟอร์มสมัคร

pc1-frontend/html/js/auth.js · บรรทัด 42–66 (ตัดบรรทัดปิดปุ่มออก)

```
registerForm.addEventListener("submit", async (e) => {
  e.preventDefault(); // ไม่ให้ฟอร์มส่งแบบเดิม หน้าจึงไม่รีเฟรช
  try {
    const data = await apiJson("/api/register", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        username: document.getElementById("username").value,
        password: document.getElementById("password").value,
        confirm: document.getElementById("confirm").value,
      }),
    });
    setFlash(data.message || "สมัครสมาชิกสำเร็จ …", "success");
    window.location.href = "login.html";
  } catch (err) {
    showFlash(err.message, "error"); // ข้อความจาก PC2 เช่น "username นี้มีคนใช้แล้ว"
  }
});
```

ภาคผนวก C4 · โค้ด

PC2 · สมัคร และล็อกอิน (hash รหัสผ่าน)

pc2-backend/app.py · บรรทัด 192–245 (ตัดมาเฉพาะส่วนสำคัญ)

```
@app.route("/api/register", methods=["POST"])
def api_register():
    … # อ่าน JSON แล้วตรวจค่า: ครบ / ยาว ≥ 4 / สองช่องตรงกัน → ไม่ผ่านตอบ 400
    if conn.execute("SELECT id FROM users WHERE username = ?", (username,)).fetchone():
        return jsonify({"error": "username นี้มีคนใช้แล้ว"}), 409
    conn.execute("INSERT INTO users (username, password_hash, is_admin, created_at) VALUES (?, ?, 0, ?)",
        (username, generate_password_hash(password), datetime.utcnow().isoformat()), "none")
    return jsonify({"message": "สมัครสมาชิกสำเร็จ กรุณาเข้าสู่ระบบ"}), 201

@app.route("/api/login", methods=["POST"])
def api_login():
    row = conn.execute("SELECT * FROM users WHERE username = ?", (username,)).fetchone()
    if row and check_password_hash(row["password_hash"], password):
        user = User(row["id"], row["username"], row["is_admin"])
        login_user(user) # ใส่ user id ลง session cookie
        return jsonify({"user": user_json(user)})
    return jsonify({"error": "username หรือ password ไม่ถูกต้อง"}), 401
```

ภาคผนวก C5 · โค้ด

PC2 · ตรวจ cookie และ /api/me

pc2-backend/app.py · บรรทัด 113–117, 159–167, 256–260

```
@login_manager.unauthorized_handler
def unauthorized(): # ไม่มี cookie หรือ cookie ปลอม → ตอบ 401 เป็น JSON
    return jsonify({"error": "กรุณาเข้าสู่ระบบก่อนใช้งาน", "code": "unauthorized"}), 401

@login_manager.user_loader
def load_user(user_id): # มี cookie → ได้ user id แล้วค้นที่ PC4
    conn = get_db()
    row = conn.execute(
        "SELECT id, username, is_admin FROM users WHERE id = ?", (user_id,)
    ).fetchone()
    if row:
        return User(row["id"], row["username"], row["is_admin"])
    return None

@app.route("/api/me")
@login_required # ทำงานก่อน api_me() ไม่ผ่าน = เรียก unauthorized()
def api_me():
    return jsonify({"user": user_json(current_user)})
```

ภาคผนวก C6 · โค้ด

PC2 · /api/generate ตรวจค่า แล้วเรียก PC3

pc2-backend/app.py · บรรทัด 454–516 (ตัดมาเฉพาะส่วนสำคัญ)

```
@app.route("/api/generate", methods=["POST"])
@login_required
def api_generate():
    … # อ่าน JSON: prompt, steps, cfg_scale, width, height, sampler, seed, checkpoint
    steps = max(1, min(steps, 50)) # บีบค่าไม่ให้ PC3 ค้าง
    width = max(64, min(width, 1024))
    wait = _generate_cooldown_left(current_user.id) # 1 ครั้ง / 3 วิ / คน
    if wait > 0:
        return jsonify({"error": f"กรุณารอสักครู่ก่อนสร้างภาพใหม่ ({wait:.1f} วินาที)"}), 429
    payload = {"prompt": prompt, "steps": steps, "sampler_name": sampler, "seed": seed, …}
    if checkpoint:
        payload["override_settings"] = {"sd_model_checkpoint": checkpoint}
    resp = requests.post(
        f"{config.FORGE_API_URL}/sdapi/v1/txt2img",
        json=payload,
        auth=config.FORGE_AUTH, # Basic Auth จาก .env
        timeout=(10, 300), # ต่อไม่ติดใน 10 วิ = PC3 ดับ (502) · รอวาดได้ 5 นาที
    )
```

ภาคผนวก C7 · โค้ด

PC2 · เก็บภาพที่ PC4 และส่งภาพผ่าน /media/

pc2-backend/app.py · บรรทัด 540–568 และ 179–186 (ตัดมาเฉพาะส่วนสำคัญ)

```
    image_bytes = base64.b64decode(images[0]) # ข้อความ base64 → byte ของ PNG
    filename = f"{current_user.id}_{int(time.time() * 1000)}.png"
    relative_path = f"outputs/{filename}"
    put_file(relative_path, image_bytes) # ส่งไฟล์ไปเก็บที่ PC4 ก่อน
    cursor = conn.execute("""INSERT INTO generations (user_id, prompt, …, image_path, created_at)
            VALUES (?, ?, …, ?, ?)""", (current_user.id, prompt, …, relative_path, …), "none")
    return jsonify({"id": cursor.lastrowid, "image_url": media_url(relative_path),
            "prompt": prompt, "seed": actual_seed})

@app.route("/media/<path:filename>")
@login_required # ภาพต้องล็อกอินก่อน
def media(filename):
    data = get_file(filename) # ขอไฟล์จาก PC4
    if data is None:
        return jsonify({"error": "ไม่พบไฟล์ภาพ"}), 404
    mimetype = mimetypes.guess_type(filename)[0] or "application/octet-stream"
    return Response(data, mimetype=mimetype)
```

ภาคผนวก C8 · โค้ด

PC2 · db_client.py ส่ง SQL และไฟล์ไป PC4

pc2-backend/db_client.py · บรรทัด 59–77 และ 124–139 (ตัด try/except ออก)

```
def execute(self, sql, params=(), fetch="all"): # ใช้เหมือน sqlite3 แต่ส่งไป PC4
    resp = requests.post(
        f"{DB_SERVER_URL}/sql",
        json={"sql": sql, "params": list(params), "fetch": fetch},
        headers={"X-DB-Token": DB_TOKEN},
        timeout=DB_TIMEOUT,
    )
    … # ต่อไม่ติด หรือตอบไม่ใช่ 200 → DatabaseError → app.py ตอบ 503
    return Result(resp.json())

def _file_request(method, rel_path, **kwargs): # ใช้กับ PUT / GET / DELETE ไฟล์ภาพ
    headers = kwargs.pop("headers", {})
    headers["X-DB-Token"] = DB_TOKEN
    url = f"{DB_SERVER_URL}/files/{quote(rel_path, safe='/')}"
    return requests.request(method, url, headers=headers, timeout=FILE_TIMEOUT, **kwargs)

def put_file(rel_path, data): # get_file ใช้ "GET" · delete_file ใช้ "DELETE"
    resp = _file_request("PUT", rel_path, data=data, headers={"Content-Type": "application/octet-stream"})
```

ภาคผนวก C9 · โค้ด

PC4 · db_server.py ตรวจสิทธิ์ แล้วรัน SQL

pc4-database/db_server.py · บรรทัด 86–208 (ตัดมาเฉพาะส่วนสำคัญ)

```
@app.before_request
def check_client(): # ทุก request ผ่านตรงนี้ก่อน (ยกเว้น /health)
    if ALLOWED_CLIENT_IPS and request.remote_addr not in ALLOWED_CLIENT_IPS:
        return jsonify({"error": f"IP {request.remote_addr} ไม่ได้รับอนุญาต…"}), 403
    if request.headers.get("X-DB-Token") != DB_TOKEN:
        return jsonify({"error": "DB token ไม่ถูกต้อง"}), 401

@app.route("/sql", methods=["POST"])
def run_sql(): # ตอบ {rows, lastrowid, rowcount}
    cur = conn.execute(sql, tuple(params))
    conn.commit() # commit ทุกคำสั่ง ฝั่ง PC2 ไม่ต้อง commit เอง

@app.route("/files/<path:rel_path>", methods=["PUT"])
def put_file(rel_path):
    full = _file_path(rel_path) # รับเฉพาะ outputs/ + safe_join กัน ../
    with open(full, "wb") as f:
        f.write(request.get_data())
    print(f"[FILE] เก็บ {rel_path}") # ขึ้นบนหน้าต่าง PC4
```

ภาคผนวก C10 · โค้ด

PC2 · /api/edit แต่งภาพ แล้วเก็บที่ PC4

pc2-backend/app.py · บรรทัด 577–648 (ตัดมาเฉพาะส่วนสำคัญ)

```
@app.route("/api/edit", methods=["POST"])
@login_required
def api_edit():
    operation = request.form.get("operation", "")
    uploaded = request.files.get("image")
    generation_id = request.form.get("generation_id")
    if uploaded and uploaded.filename: # อัปโหลดเอง: ภาพมากับคำขอเลย
        img = cv2.imdecode(np.frombuffer(uploaded.read(), dtype=np.uint8), cv2.IMREAD_COLOR)
    elif generation_id: # เลือกจากประวัติ: ต้องเป็นของ user นี้
        gen_row = conn.execute("SELECT * FROM generations WHERE id = ? AND user_id = ?", …).fetchone()
        raw_bytes = get_file(gen_row["image_path"]) # ภาพต้นฉบับอยู่ที่ PC4
        img = cv2.imdecode(np.frombuffer(raw_bytes, dtype=np.uint8), cv2.IMREAD_COLOR)
    result, params = image_ops.apply_operation(img, operation, request.form)
    ok, png = cv2.imencode(".png", result) # ผลลัพธ์ → PNG ในหน่วยความจำ
    relative_path = f"outputs/edits/edit_{current_user.id}_{uuid.uuid4().hex}.png"
    put_file(relative_path, png.tobytes()) # เก็บที่ PC4
    conn.execute("""INSERT INTO edits (…, params, result_image, created_at) VALUES …""", …)
    return jsonify({"image_url": media_url(relative_path), "operation_label": …, …})
```

ภาคผนวก C11 · โค้ด

PC2 · admin ลบรายการ

pc2-backend/app.py · บรรทัด 318–344

```
@app.route("/api/admin/delete/<int:record_id>", methods=["POST"])
@login_required
def api_admin_delete(record_id):
    if not current_user.is_admin: # เช็คซ้ำฝั่ง server ไม่เชื่อแค่หน้าเว็บ
        return jsonify({"error": "หน้านี้สำหรับ admin เท่านั้น"}), 403
    record = conn.execute("SELECT * FROM generations WHERE id = ?", (record_id,)).fetchone()
    if not record:
        return jsonify({"error": "ไม่พบรายการนี้ (อาจถูกลบไปแล้ว)"}), 404
    # ลบ record ที่ PC4 ก่อน — ถ้า DB ลบไม่สำเร็จจะได้ไม่เสียไฟล์ภาพไปฟรี ๆ
    result = conn.execute("DELETE FROM generations WHERE id = ?", (record_id,), "none")
    if result.rowcount < 1:
        return jsonify({"error": "ลบข้อมูลในฐานข้อมูล (PC4) ไม่สำเร็จ"}), 500
    try:
        delete_file(record["image_path"]) # แล้วค่อยสั่ง PC4 ลบไฟล์ภาพ
    except DatabaseError as e:
        app.logger.warning("Could not delete image file %s: %s", record["image_path"], e)
    return jsonify({"message": "ลบรายการนี้แล้ว"})
```

ภาคผนวก C12 · โค้ด

PC2 + PC4 · config.py และไฟล์ .env

pc2-backend/.env.example · pc2-backend/config.py บรรทัด 35–51 · ค่าจริงไม่ได้ใส่ไว้ในสไลด์

```
# pc2-backend/.env
SECRET_KEY=<สุ่ม 64 ตัวอักษร ใช้เซ็น cookie>
FORGE_API_USER=<ต้องตรงกับ --api-auth ของ PC3>
FORGE_API_PASS=<…>
DB_TOKEN=<ต้องตรงกับ pc4-database/.env>
FLASK_DEBUG=false
GENERATE_COOLDOWN_SECONDS=3

# pc2-backend/config.py
FORGE_API_URL = "http://172.20.56.108:7860"
FORGE_AUTH = (FORGE_API_USER, FORGE_API_PASS) if FORGE_API_USER else None
DB_SERVER_URL = "http://172.20.56.113:5100"
DB_TOKEN = os.environ.get("DB_TOKEN", "")
if not DB_TOKEN:
    raise SystemExit("[X] ยังไม่ได้ตั้งค่า DB_TOKEN …") # ไม่มี token = ไม่ยอมเปิด

# pc4-database/.env DB_TOKEN=<ค่าเดียวกับ PC2>
```